

CS4225/CS5425 Big Data Systems for Data Science

Streaming Processing

Ai Xin
School of Computing
National University of Singapore
aixin@comp.nus.edu.sg



Today's Plan

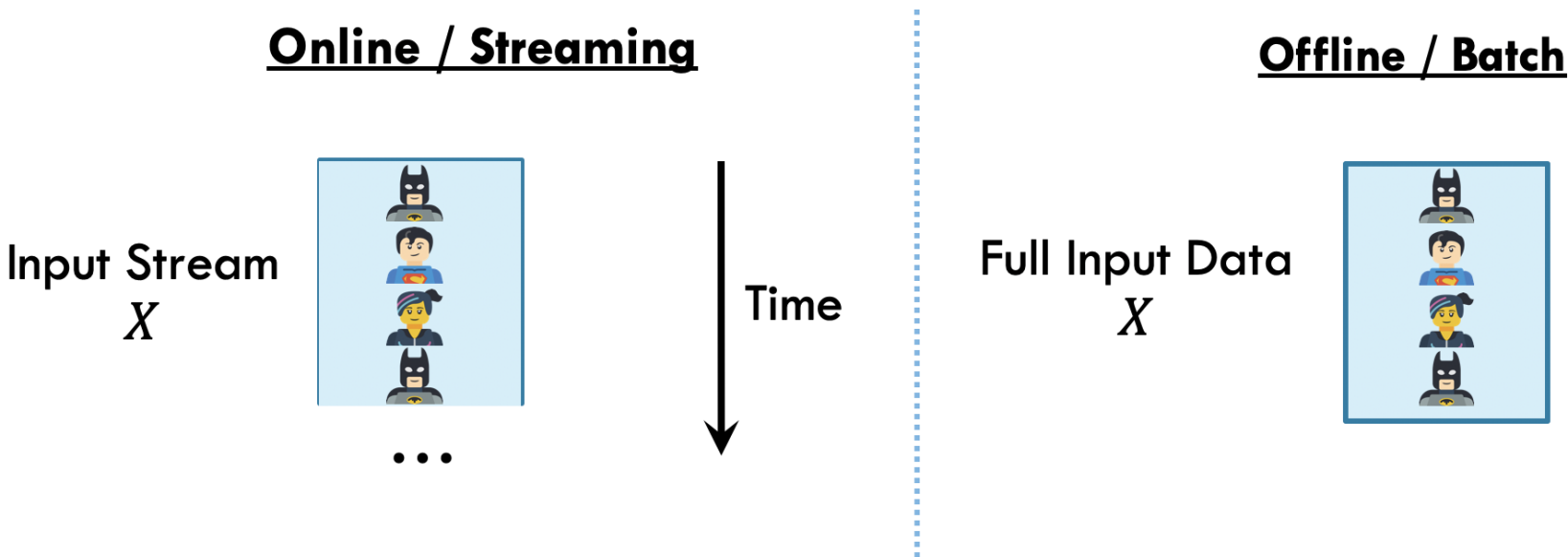
- Introduction

- Spark

- Flink

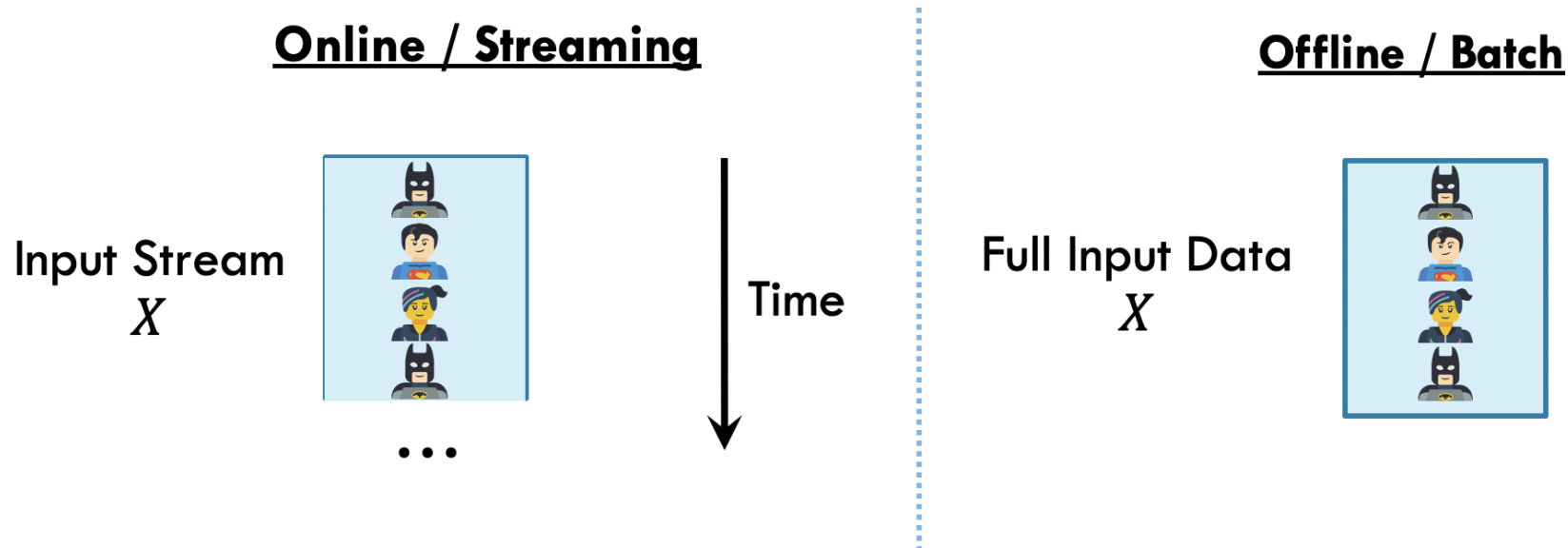
Streams: Motivation

- In many settings, the data is **arriving over time**; it is not received all at once
 - Streaming (or “online”) approaches are designed to process their input **as it is received**
 - This is in contrast to offline or batch approaches that operate on the full dataset, all at once



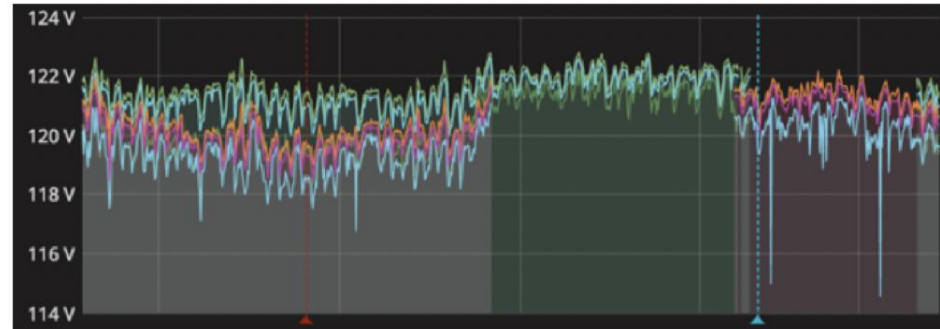
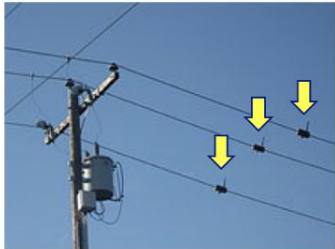
Streaming Data

- Input elements enter at a rapid rate from **input ports** (e.g. receiving data from a sensor, from a TCP connection, from a file stream, or from a message queue)
 - Elements of the stream are sometimes referred to as ‘tuples’
 - The stream is potentially infinite; **the system cannot store the entire stream accessibly** (due to limited memory)



Example Applications

- In many real settings, data is **high volume** and **constantly arriving over time**



Sensor data (industrial, environmental, medical, etc)

Example Applications

- In many real settings, data is **high volume** and **constantly arriving over time**



Online user activity data

Example Applications

- In many real settings, data is **high volume** and **constantly arriving over time**



Financial Transactions
(e.g. credit card fraud detection)



Stock Trades Data
(e.g. trades, orders, prices)

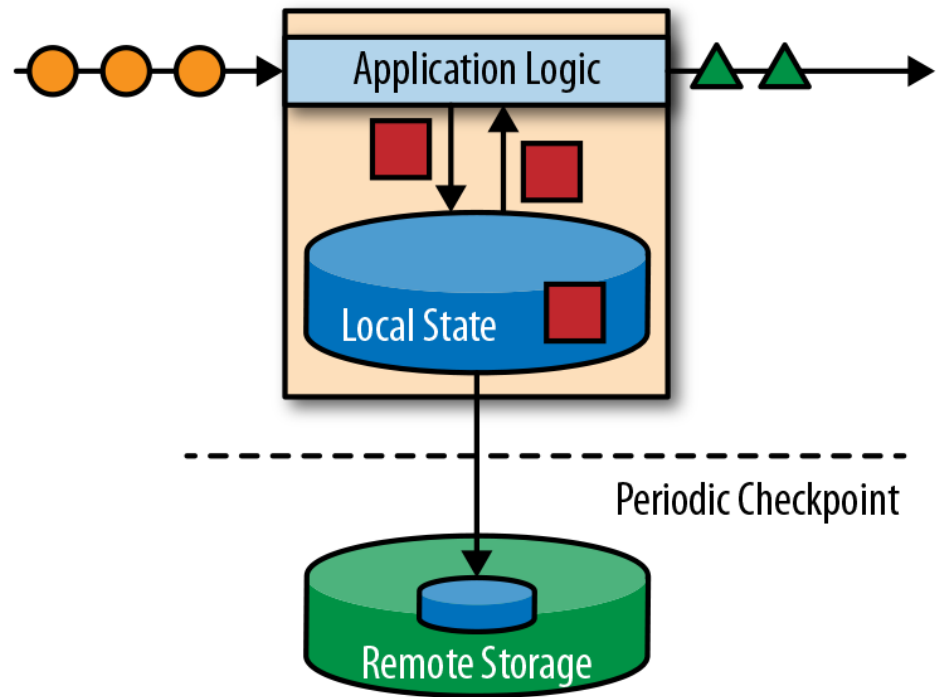
02 Sep 2020	OMO STORE		S\$3.30
02 Sep 2020	FACEBK	V2 650-5434800 CA USD2.00	S\$2.81
02 Sep 2020	FACEBK	650-5434800 CA USD3.00	S\$4.22
02 Sep 2020	FACEBK	650-5434800 CA USD2.00	S\$2.81
02 Sep 2020	FACEBK	650-5434800 CA USD3.00	S\$4.22
02 Sep 2020	FACEBK	650-5434800 CA USD5.00	S\$7.03
02 Sep 2020	FACEBK	650-5434800 CA USD50.00	S\$70.45
02 Sep 2020	FACEBK	650-5434800 CA USD10.00	S\$14.10
03 Sep 2020	FACEBK	650-5434800 CA USD50.00	S\$70.45
03 Sep 2020	FACEBK	650-5434800 CA USD75.00	S\$105.84
03 Sep 2020	FACEBK	650-5434800 CA USD1.33	
03 Sep 2020	OMO STORE		S\$3.00
03 Sep 2020	GRAB*GRAB*	SINGAPORE SG	S\$50.00

Do you need any help?



Stateful Stream Processing

- not just perform trivial record-at-a-time transformations
- the ability to store and access intermediate data
- state can be stored and accessed in many different places including program variables, local files, or embedded or external databases
- Exactly-Once processing guarantees



Today's Plan

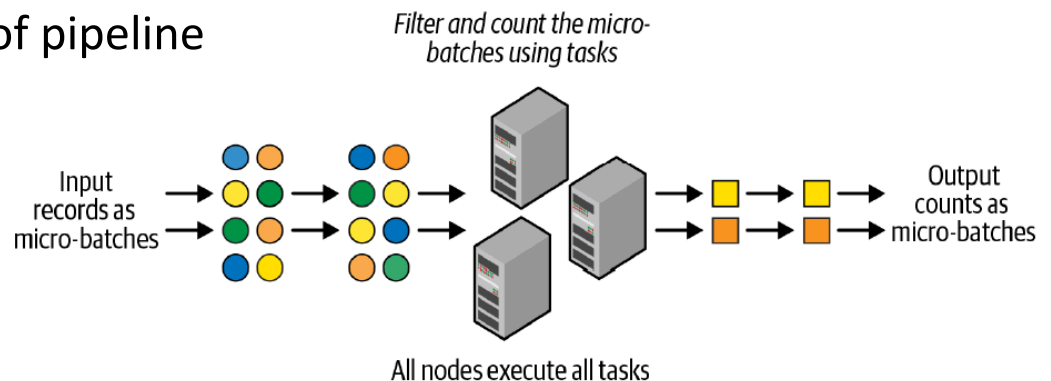
- Introduction

- Spark

- Flink

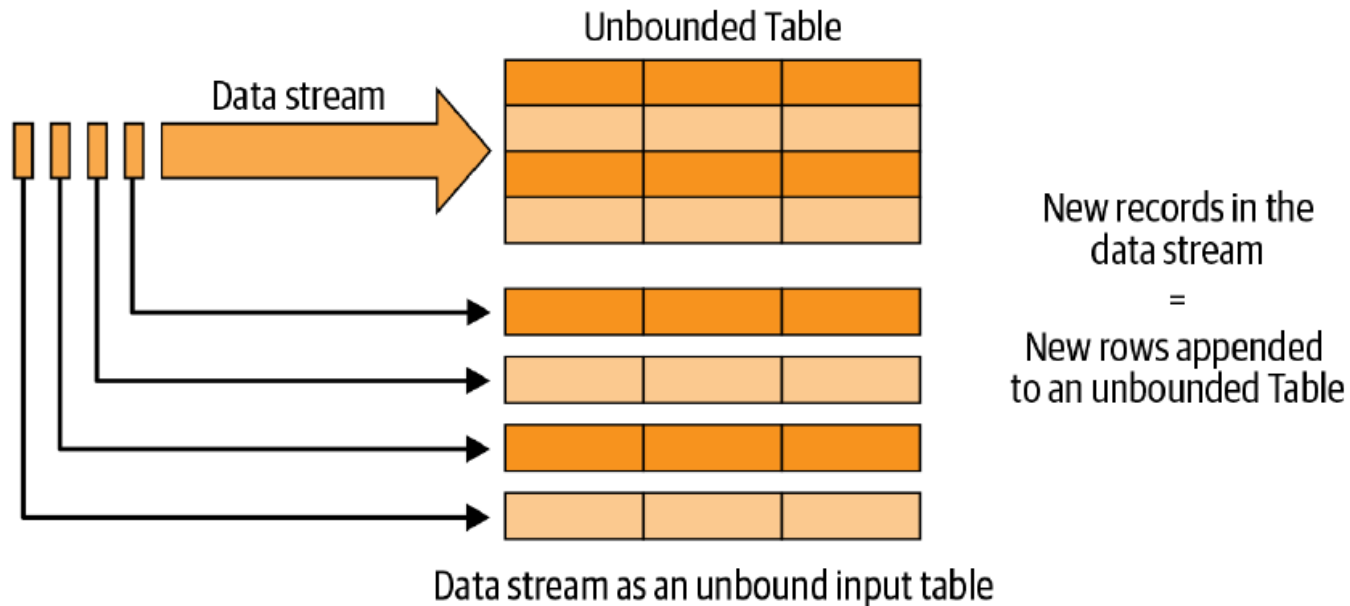
Micro-Batch Stream Processing

- Spark Structured Streaming uses a micro-batch processing model
 - divides the data from the input stream into micro batches
 - each batch is processed in the Spark cluster in a distributed manner
 - small deterministic tasks generate the output in micro-batches
- Advantages
 - quickly and efficiently recover from failures
 - deterministic nature ensures end-to-end exactly-once processing guarantees
- Disadvantages: latencies of a few seconds
 - This is actually OK for many applications
 - Application may incur more than a few seconds delay in other parts of pipeline

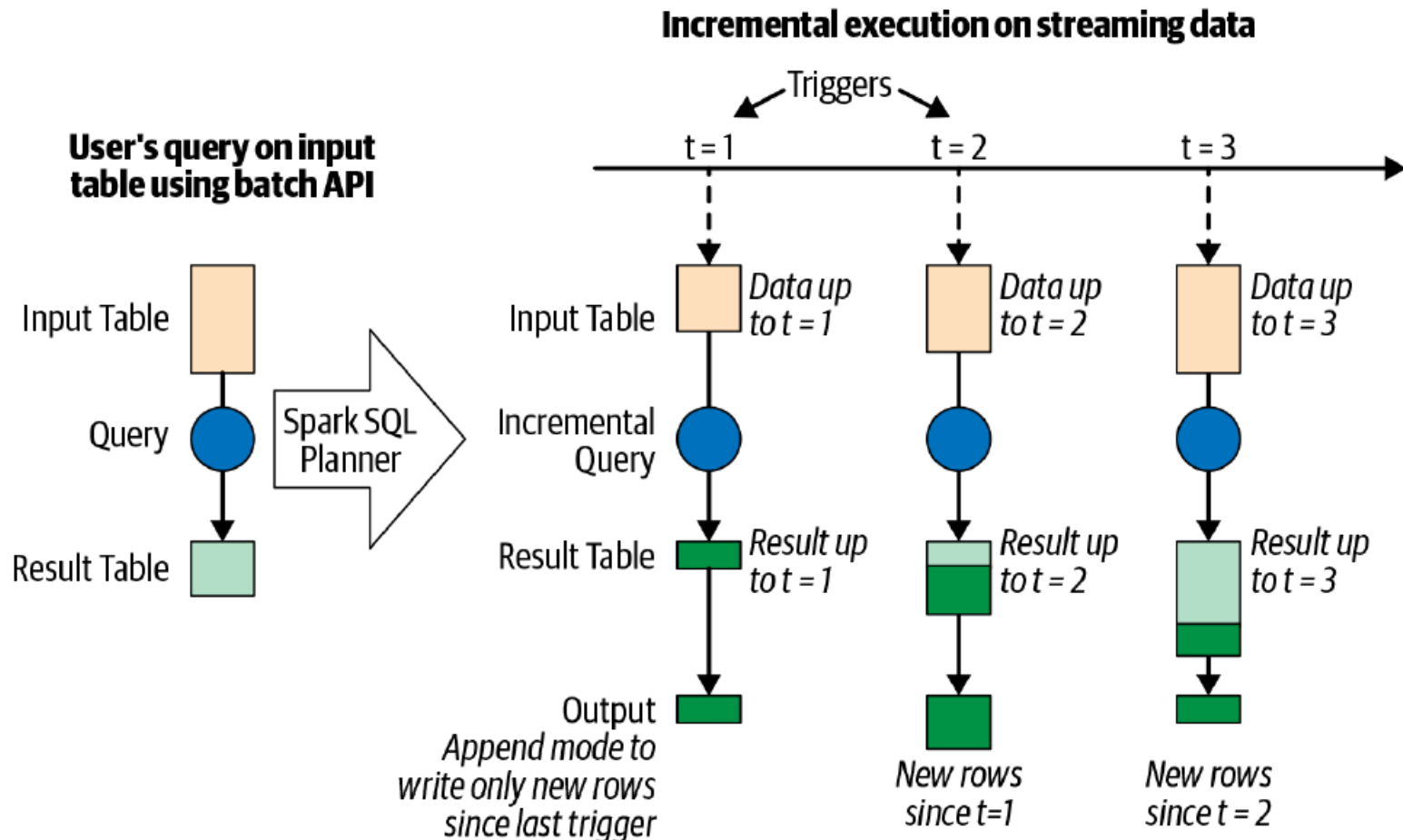


The Philosophy of Structured Streaming

- For developers, writing stream processing pipelines should be as easy as writing batch pipelines.
 - A single, unified programming model and interface for batch and stream processing
 - A broader definition of stream processing
- The Structured Streaming programming model: data stream as an unbounded table



The Structured Streaming processing model



Users express query on streaming data using a batch-like API and Structured Streaming incrementalizes them to run on streams.

Output mode

- Append Mode

- Only the new rows appended to the result table since the last trigger will be written to the external storage. This is applicable only in queries where existing rows in the result table cannot change.

- Update Mode

- Only the rows that were updated in the result table since the last trigger will be written to external storage. This mode works for output sinks that can be updated in place, such as a MySQL table.

- Complete Mode

- The entire updated result table will be written to external storage.

Five Steps to Define a Streaming Query

- Step 1: Define input sources
- Step 2: Transform data
- Step 3: Define output sink and output mode
 - Output writing details
 - where and how to write the output
- Step 4: Specify processing details
 - Triggering details: when to trigger the results and processing of newly available streaming data.
 - Checkpoint Location: store the streaming query process info for failure recovery
- Step 5: Start the query

Practical 3: Spark Streaming

Practical 3: Spark Streaming

This notebook provides a structure streaming example using Spark.

Source: <https://github.com/databricks/Spark-The-Definitive-Guide>

Cmd 2

```
1 spark.conf.set("spark.sql.shuffle.partitions", 5)
```

Command took 0.11 seconds -- by aixin@comp.nus.edu.sg at 2/13/2023, 4:24:58 PM on Test

Cmd 3

```
1 static = spark.read.json("/databricks-datasets/definitive-guide/data/activity-data/")
2 dataSchema = static.schema
3
```

▶ (3) Spark Jobs

▶ static: pyspark.sql.dataframe.DataFrame = [Arrival_Time: long, Creation_Time: long ... 8 more fields]

Command took 38.53 seconds -- by aixin@comp.nus.edu.sg at 2/13/2023, 4:25:06 PM on Test

Cmd 4

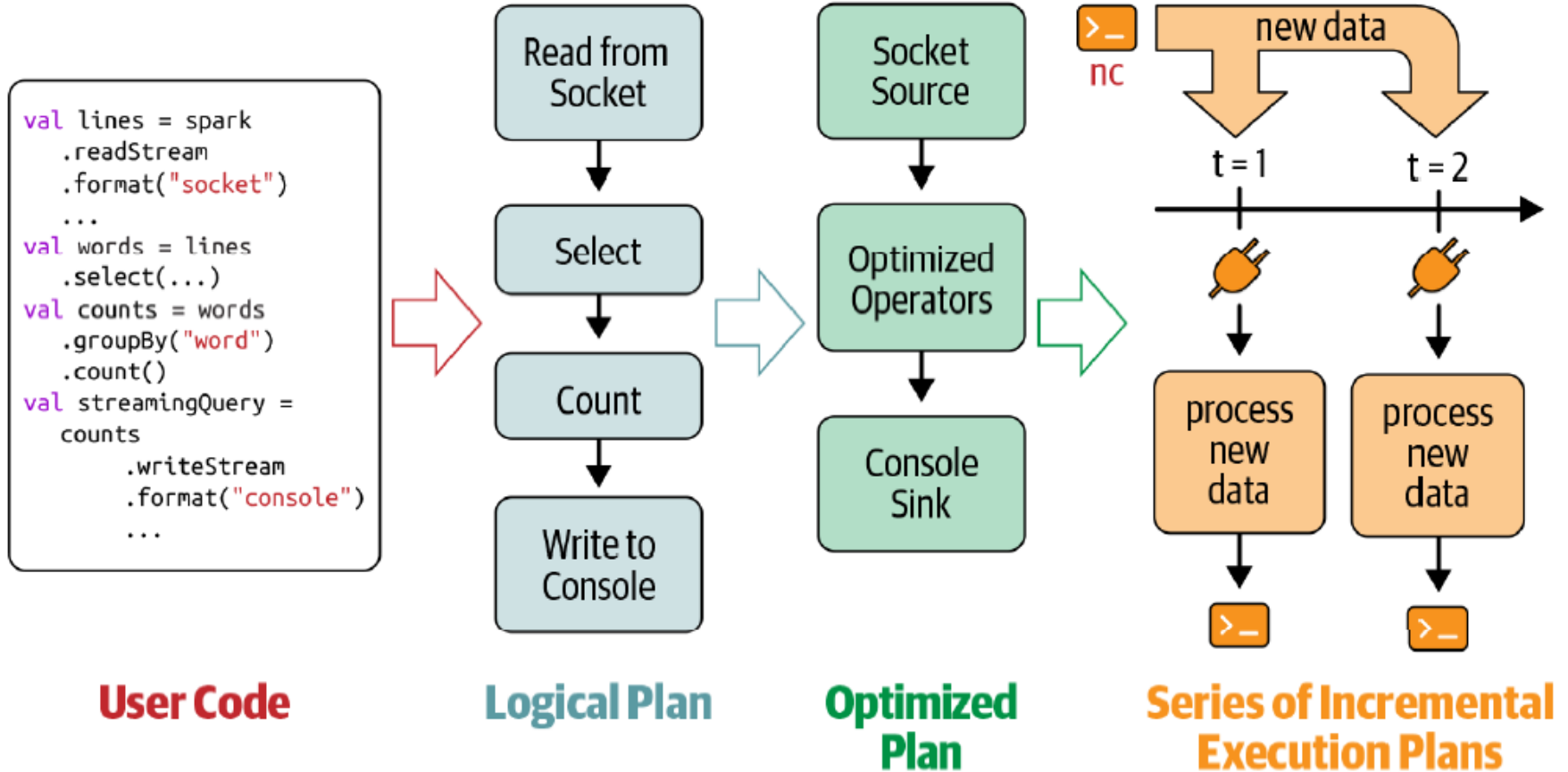
```
1 streaming = spark.readStream.schema(dataSchema).option("maxFilesPerTrigger", 1)\
2 | .json("/databricks-datasets/definitive-guide/data/activity-data")
3
```

▶ streaming: pyspark.sql.dataframe.DataFrame = [Arrival_Time: long, Creation_Time: long ... 8 more fields]

Command took 0.52 seconds -- by aixin@comp.nus.edu.sg at 2/13/2023, 4:03:19 PM on Test

Cmd 5

Incremental execution of streaming queries



Data Transformation

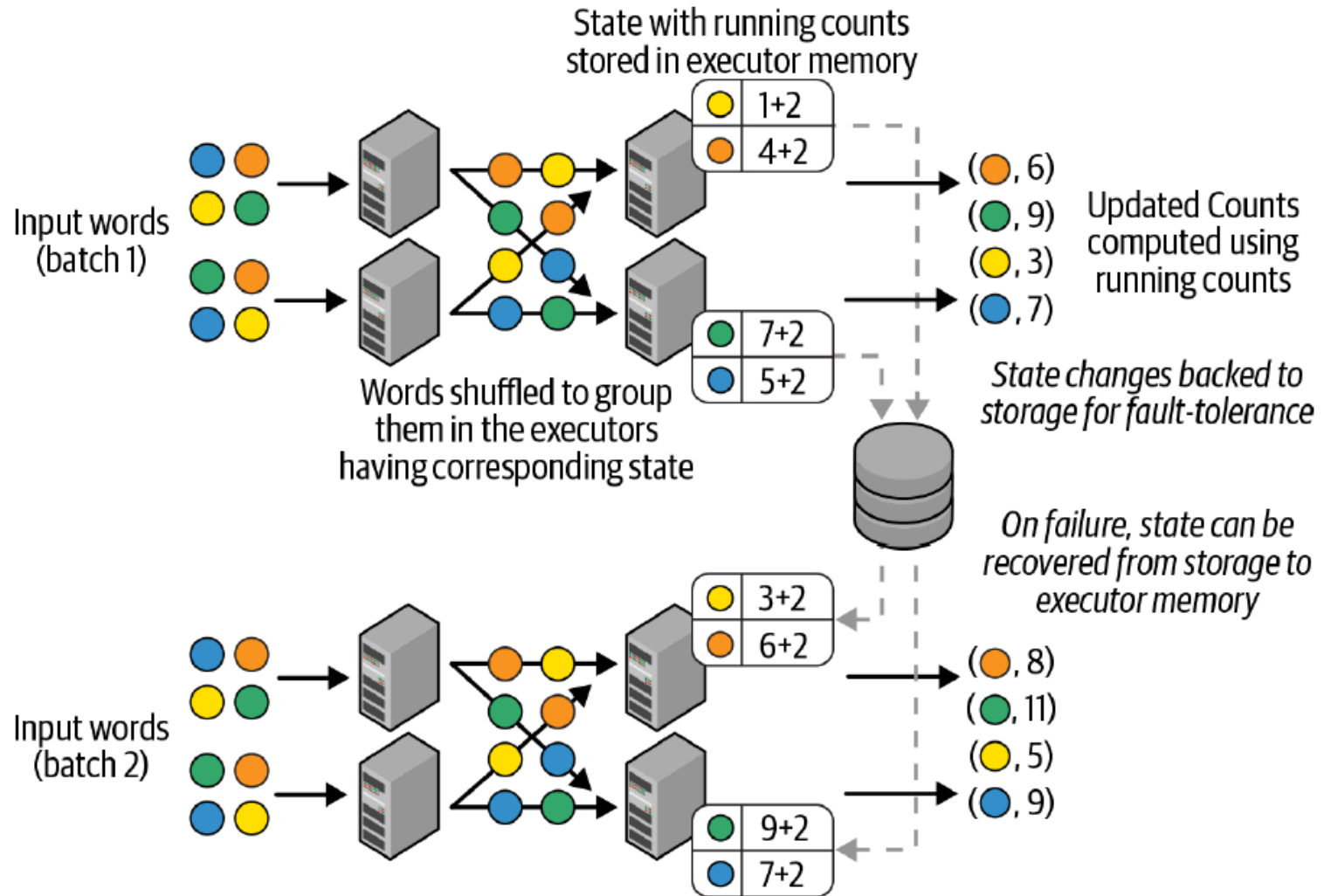
○ Stateless Transformation

- Process each row individually without needing any information from previous rows
- Projection operations: `select()`, `explode()`, `map()`, `flatMap()`
- Selection operations: `filter()`, `where()`

○ Stateful Transformation

- A simple example: `DataFrame.groupBy().count()`
- In every micro-batch, the incremental plan adds the count of new records to the previous count generated by the previous micro-batch
- The partial count communicated between plans is the state
- The state is maintained in the memory of the Spark executors and is checkpointed to the configured location to tolerate failures.

Distributed state management in Structured Streaming



Stateful Streaming Aggregations

- Aggregations Not Based on Time

- Global aggregations

```
runningCount = sensorReadings.groupBy().count()
```

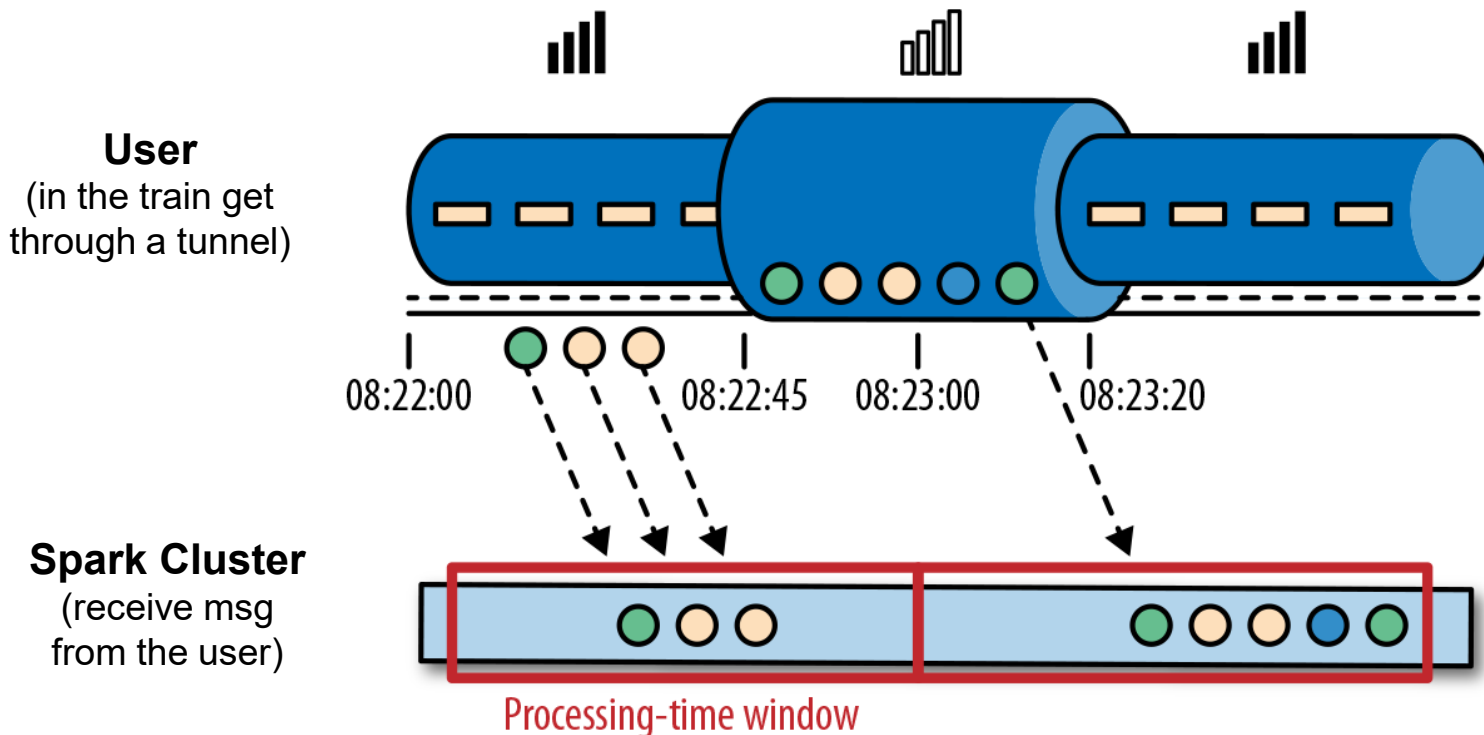
- Grouped aggregations

```
baselineValues = sensorReadings.groupBy("sensorId").mean("value")
```

- All built-in aggregation functions in DataFrames are supported
 - `sum()`, `mean()`, `stddev()`, `countDistinct()`, `collect_set()`, `approx_count_distinct()`, and etc.

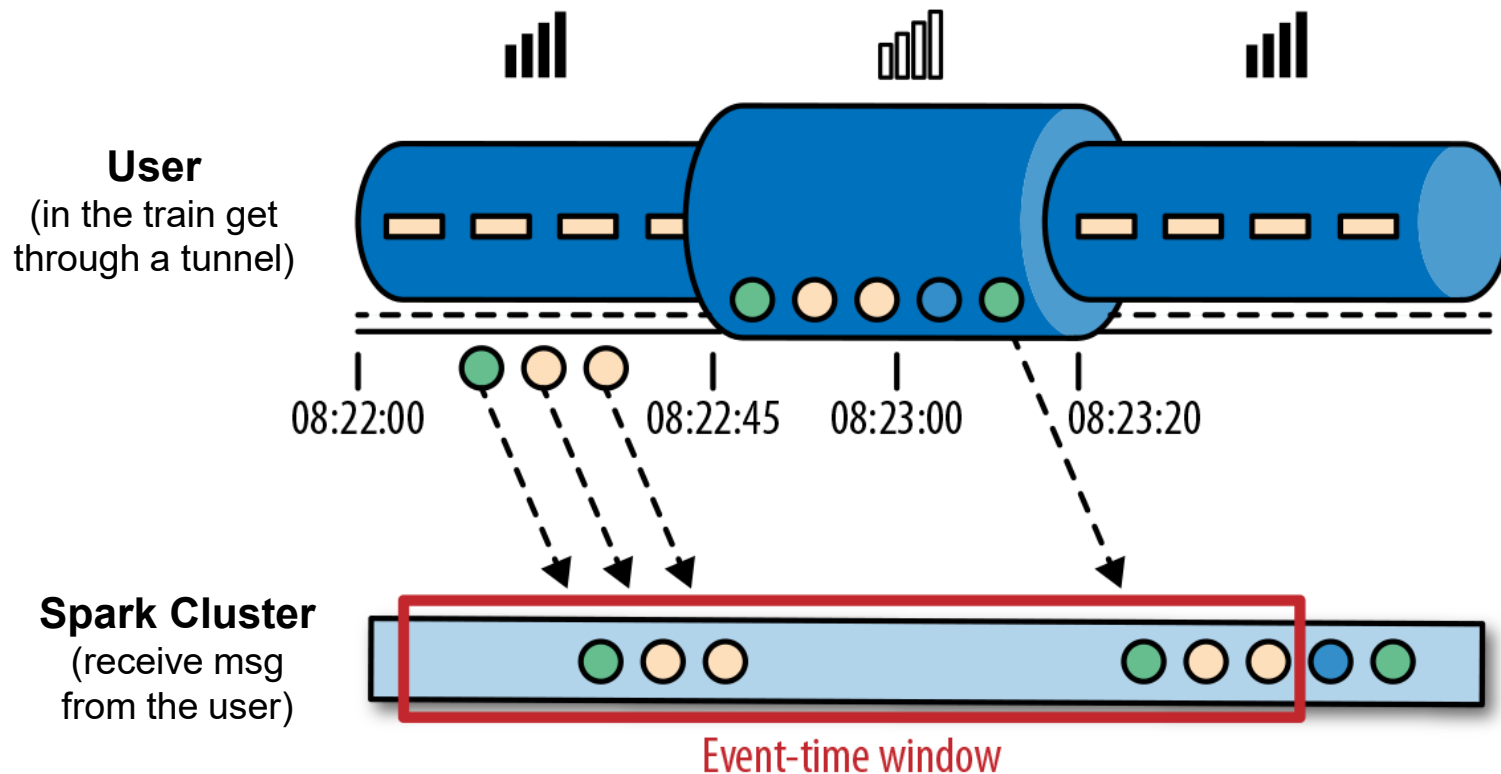
Time Semantics

- Processing Time: the time of stream processing machine



Time Semantics

- Event Time: the time an event actually happened



Time Semantics

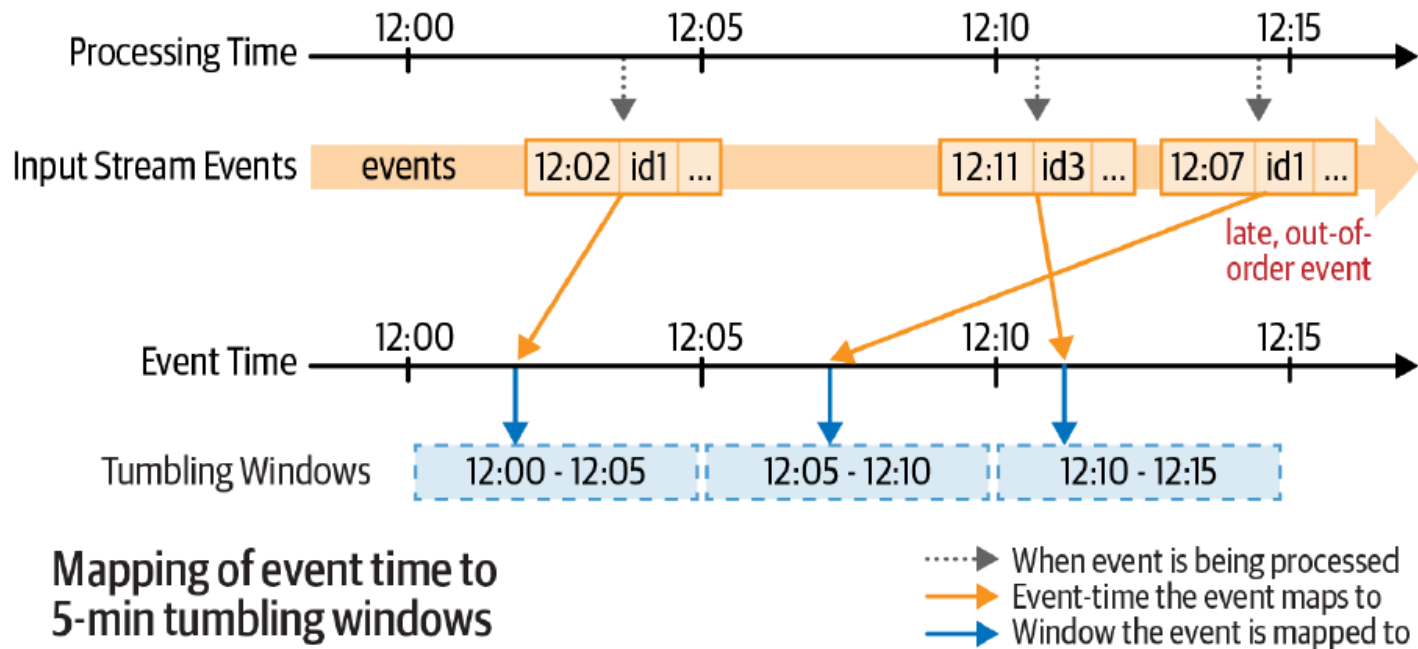
- Event time completely decouples the processing speed from the results.
- Operations based on event time are predictable and their results are deterministic.
- An event time window computation will yield the same result no matter how fast the stream is processed or when the events arrive at the operator.
- how long do we have to wait before we can be certain that we have received all events that happened before a certain point of time?

watermarks

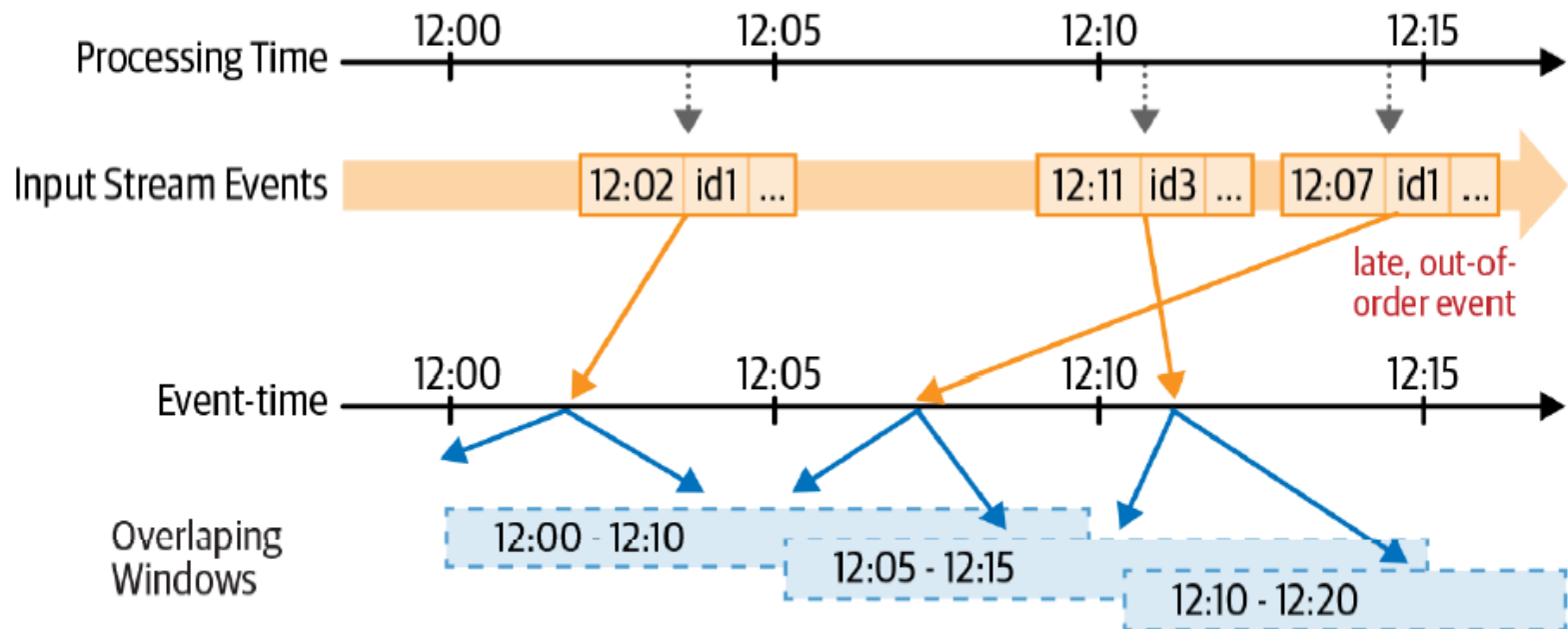
Stateful Streaming Aggregations

○ Aggregations with Event-Time Windows

```
(sensorReadings  
  .groupBy("sensorId", window("eventTime", "5 minute"))  
  .count())
```

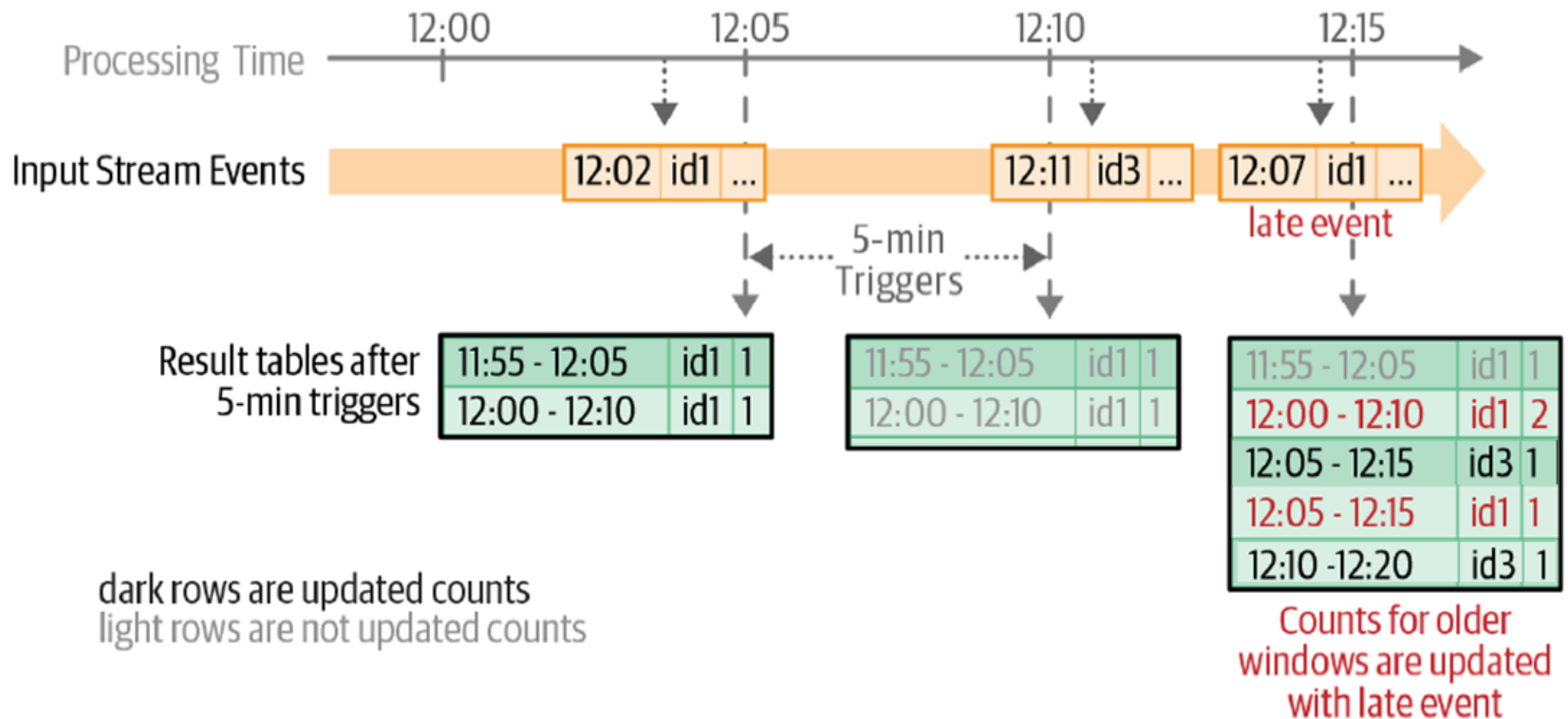


```
(sensorReadings
  .groupBy("sensorId", window("eventTime", "10 minute", "5 minute"))
  .count())
```



Mapping of event time to overlapping windows of length 10 mins and sliding interval 5 mins

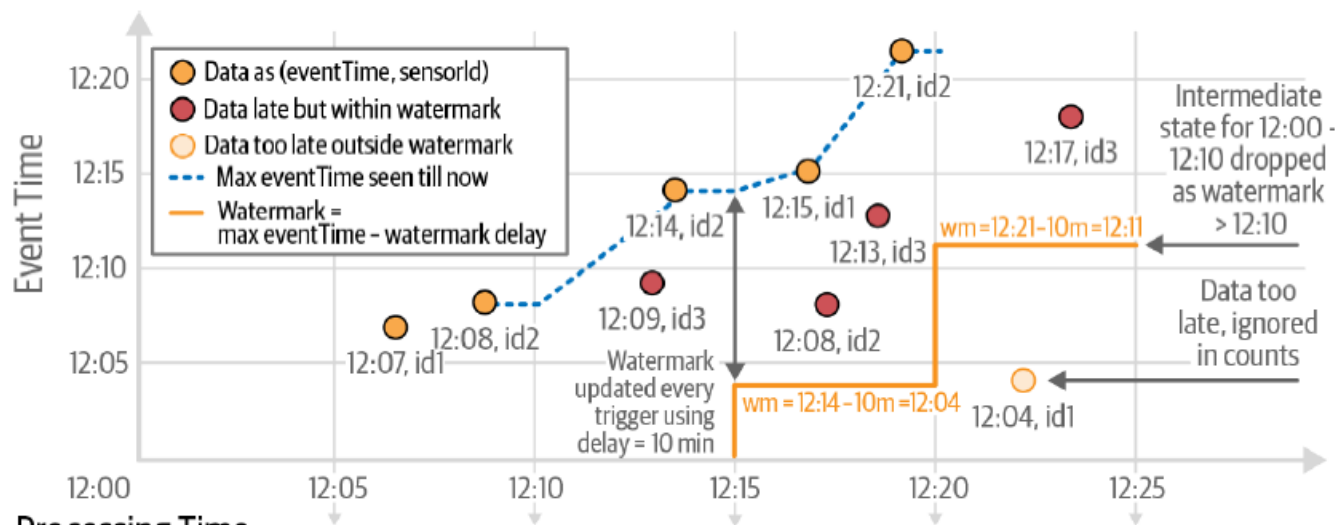
- Updated counts in the result table after each five-minute trigger



○ Handling Late Data with Watermarks

(sensorReadings

```
.withWatermark("eventTime", "10 minutes")
.groupBy("sensorId", window("eventTime", "10 minutes", "5 minutes"))
.count()
```



Processing Time
with 5 min triggers

12:00 - 12:10	id1	1
12:00 - 12:10	id2	1
12:05 - 12:15	id1	1
12:05 - 12:15	id2	1

Result Tables after each trigger

12:00 - 12:10	id1	1
12:00 - 12:10	id2	1
12:00 - 12:10	id3	1
12:05 - 12:15	id1	1
12:05 - 12:15	id2	2
12:05 - 12:15	id3	1
12:10 - 12:20	id2	1

dark rows
are updated
counts

12:00 - 12:10	id1	1
12:00 - 12:10	id2	2
12:00 - 12:10	id3	1
12:05 - 12:15	id1	2
12:05 - 12:15	id2	3
12:05 - 12:15	id3	2
12:10 - 12:20	id2	1
12:10 - 12:20	id1	1
12:10 - 12:20	id3	1
...		

12:00 - 12:10	id1	1
12:00 - 12:10	id2	2
12:00 - 12:10	id3	1
12:05 - 12:15	id1	2
12:05 - 12:15	id2	3
12:05 - 12:15	id3	2
12:10 - 12:20	id2	1
12:10 - 12:20	id1	1
12:10 - 12:20	id3	2
...		

Table *not*
updated with
too late data
(12:04, id1)

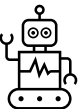
Table updated
with late data
(12:17, id3)

Watermarking in
Windowed Grouped Counts

Performance Tuning

- Besides tuning Spark SQL engine, a few other considerations
 - Cluster resource provisioning appropriately to run 24/7
 - Number of partitions for shuffles to be set much lower than batch queries
 - Setting source rate limits for stability
 - Multiple streaming queries in the same Spark application

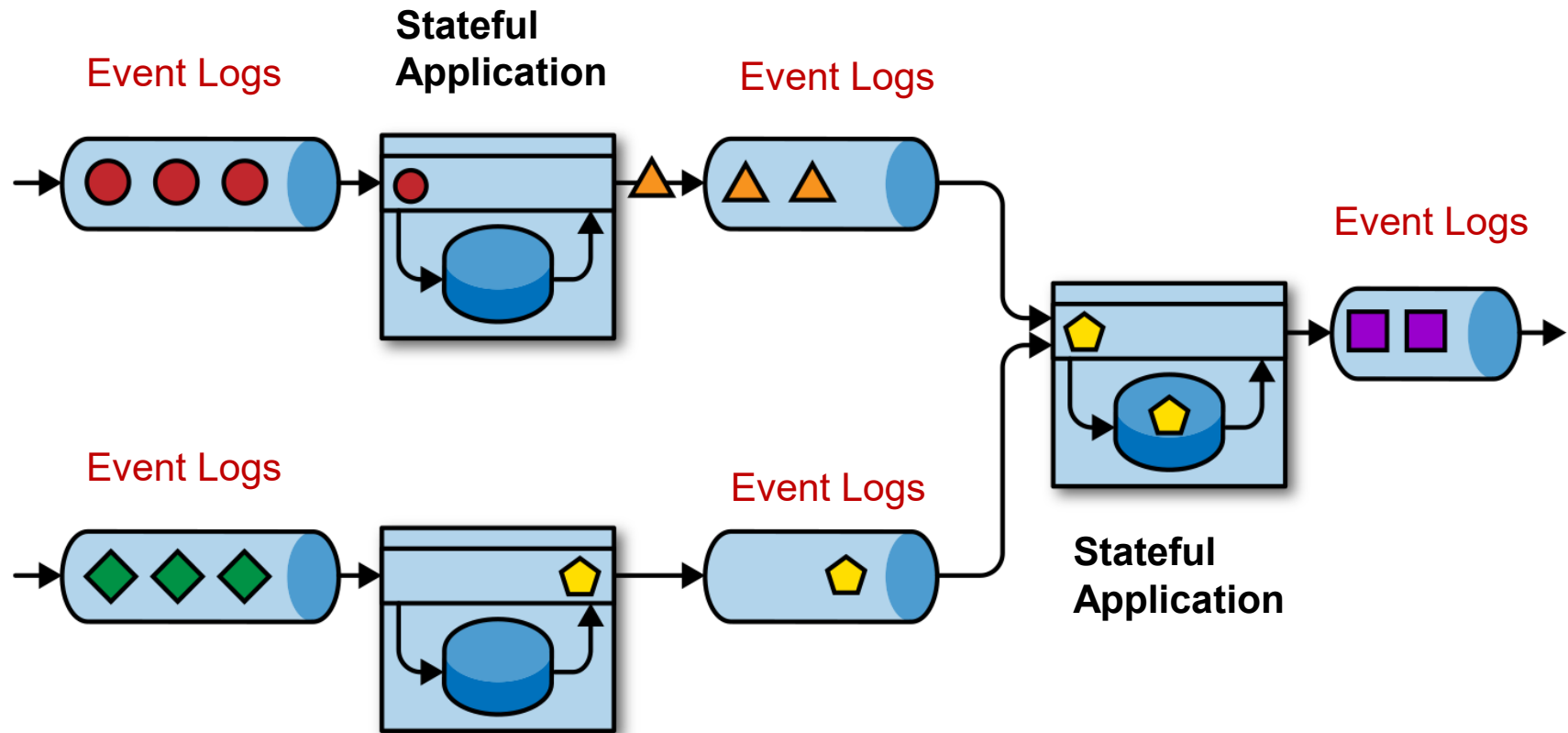
Take-away in the AI Era

- (Why Spark Stream design still require humans)
-  **What AI cannot decide for you (yet)**
 - Business-Critical Trade-offs (Latency vs. Cost)
 - Is it worth spending an extra \$5,000/month to move from 10-second micro-batches to 1-second micro-batches mode?
 - Service Level Agreements
 - Determining if you need **Exactly-Once** streaming processing (crucial for financial transactions) or if **At-Least-Once** (acceptable for simple log monitoring) is enough.
-  **What AI can assist you**
 - Migrate from legacy stream codes to structured streaming codes (e.g. Spark dataframes, Flink and etc.)
 - Performance tuning and debugging

Today's Plan

- Introduction
- Spark
- Flink

Event-driven streaming application

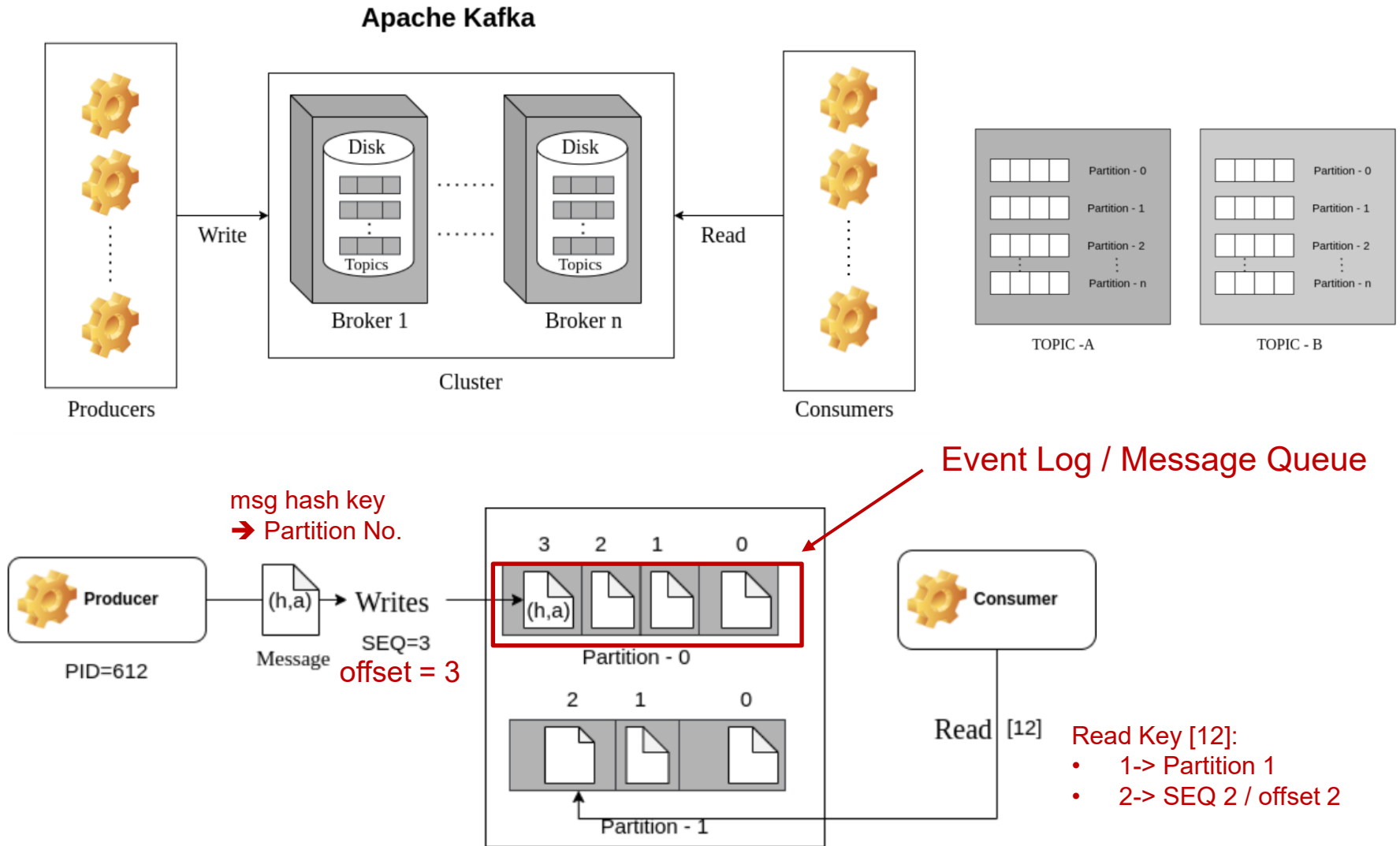


**Stateful
Application**

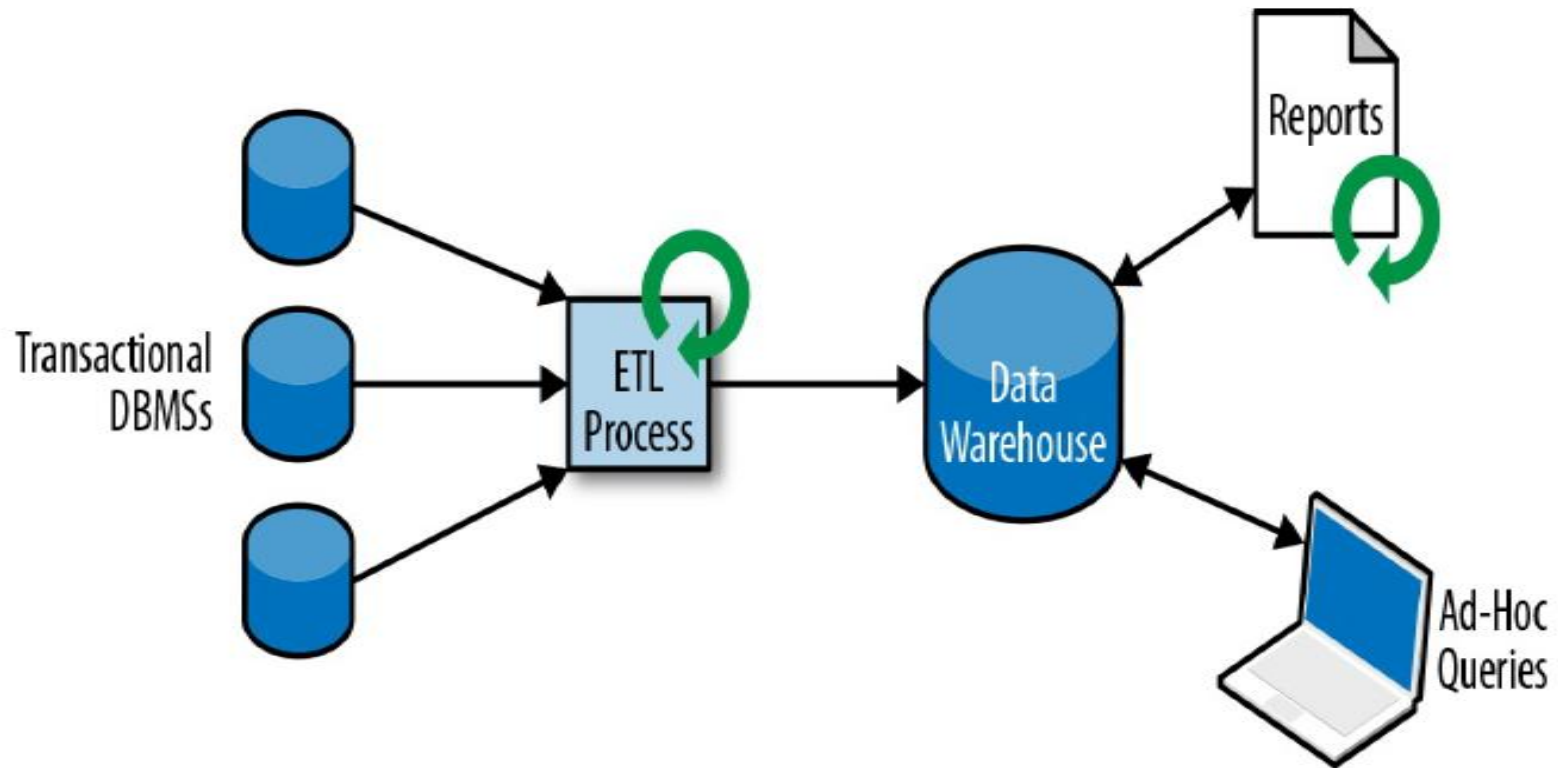
Event Logs

- decouple senders and receivers,
- provide asynchronous, nonblocking event transfer
- exactly-once consistency

Event Logs: Kafka



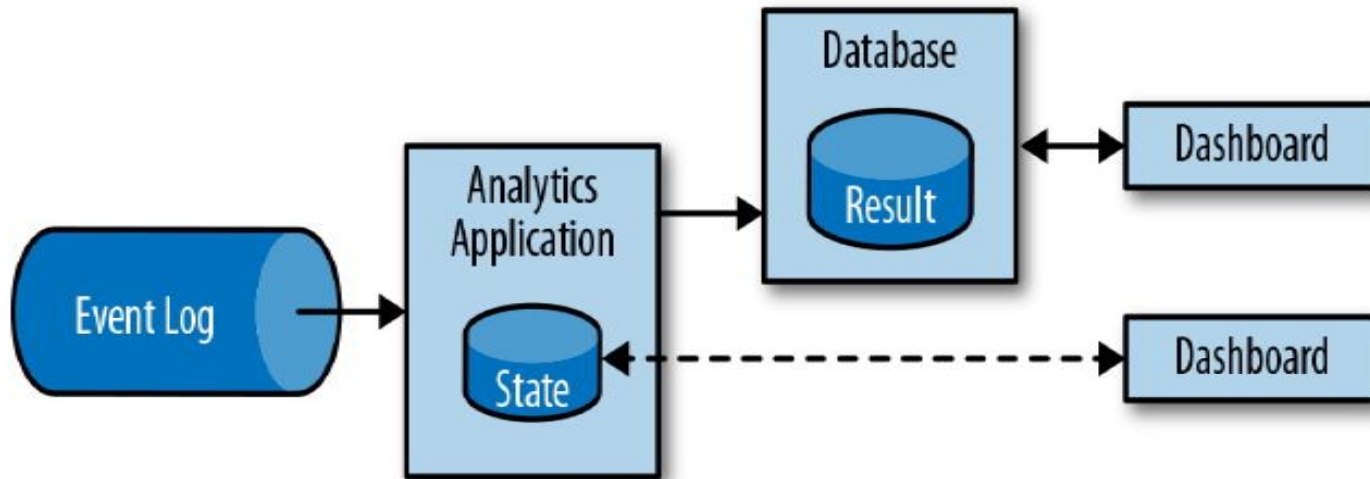
A traditional data warehouse architecture for data analytics



Traditional analytics pipelines consist of several individual components such as an ETL process, a storage system and etc.

A Streaming Analytics Application

a stream processor takes care of all the processing steps, including event ingestion, continuous computation including state maintenance, and updating the results.

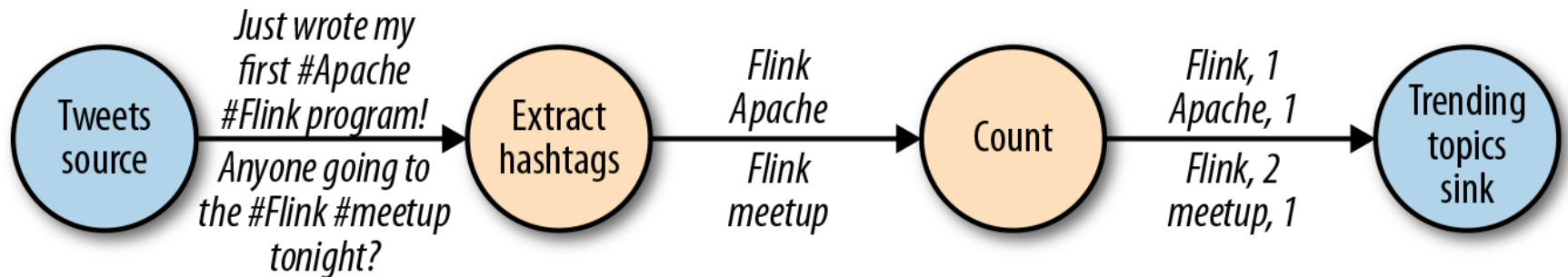


Much shorter time needed for an event to be incorporated into an analytics result !

Dataflow Model

○ Dataflow Graph

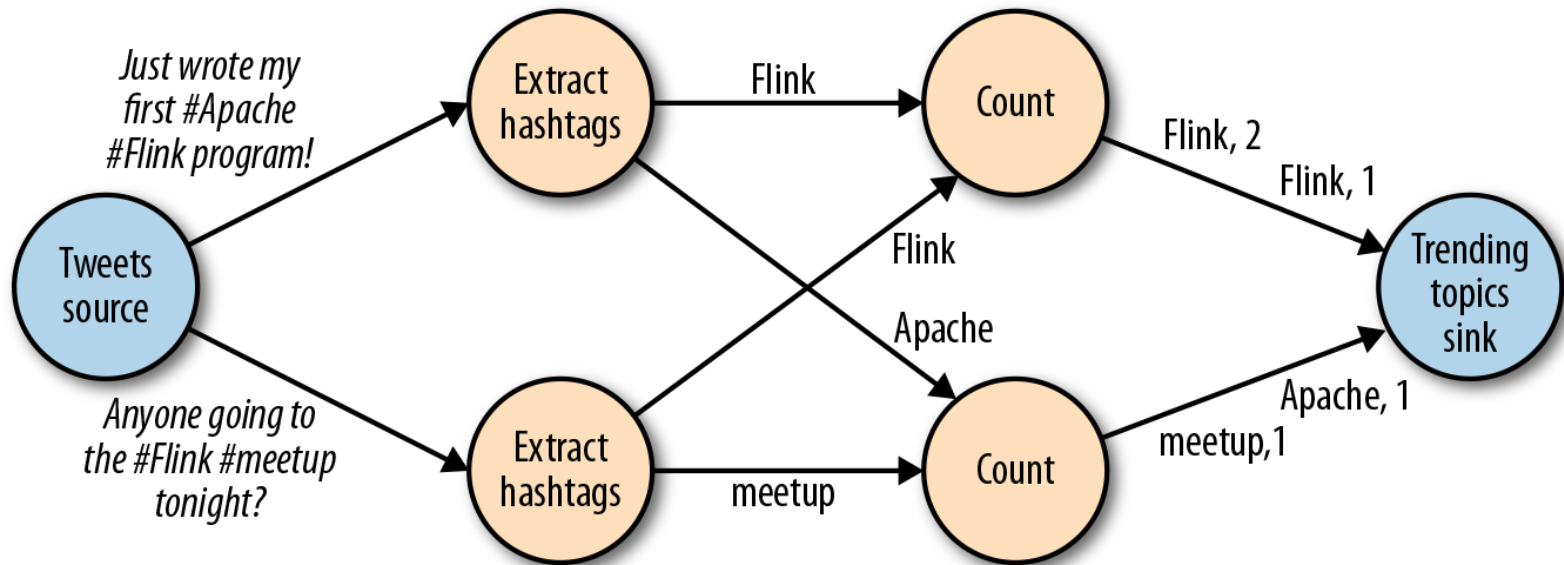
- A logical dataflow graph to continuously count hashtags



nodes represent operators and
edges denote data dependencies

Dataflow Model

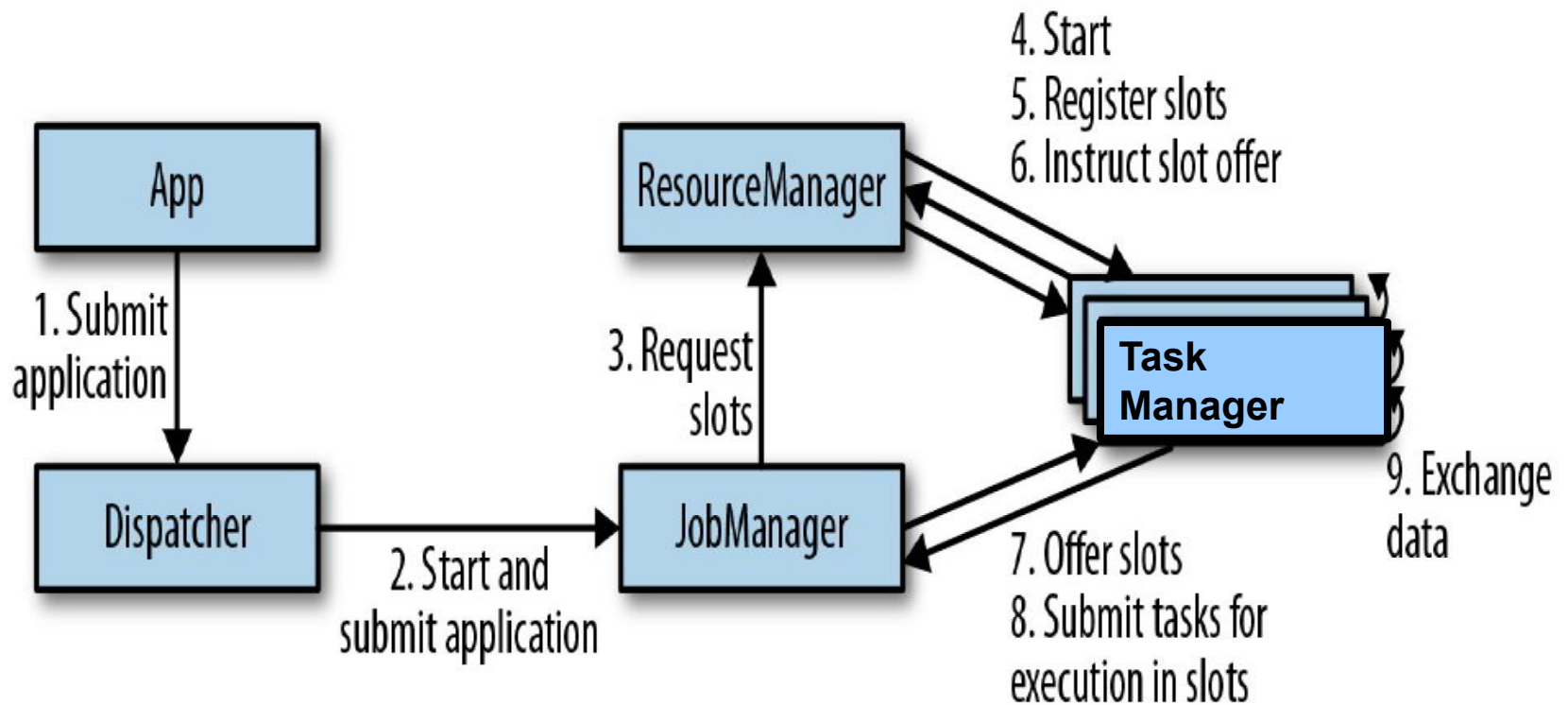
- A physical dataflow plan for counting hashtags



nodes represent tasks

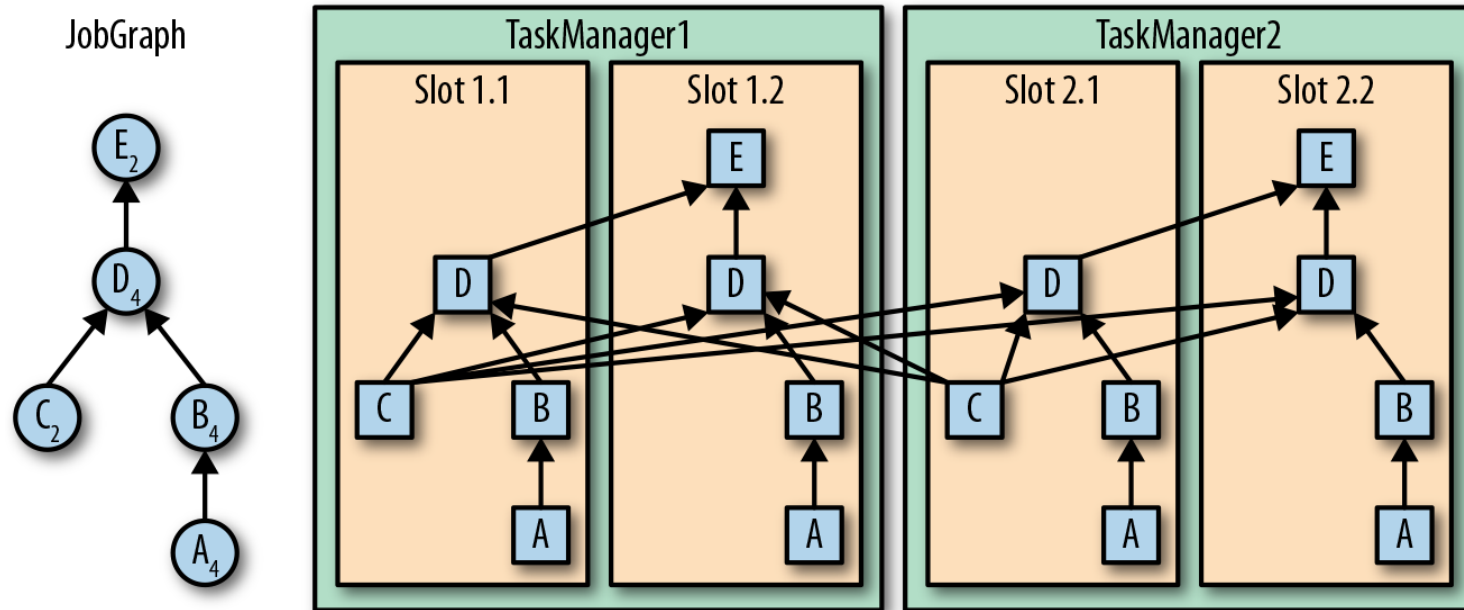
System Architecture

- Flink is a distributed system for stateful parallel data stream processing.



Task Execution

- A TaskManager can execute several tasks at the same time:
 - tasks of the same operator (data parallelism)
 - tasks of different operators (task parallelism)
 - tasks from a different application (job parallelism)



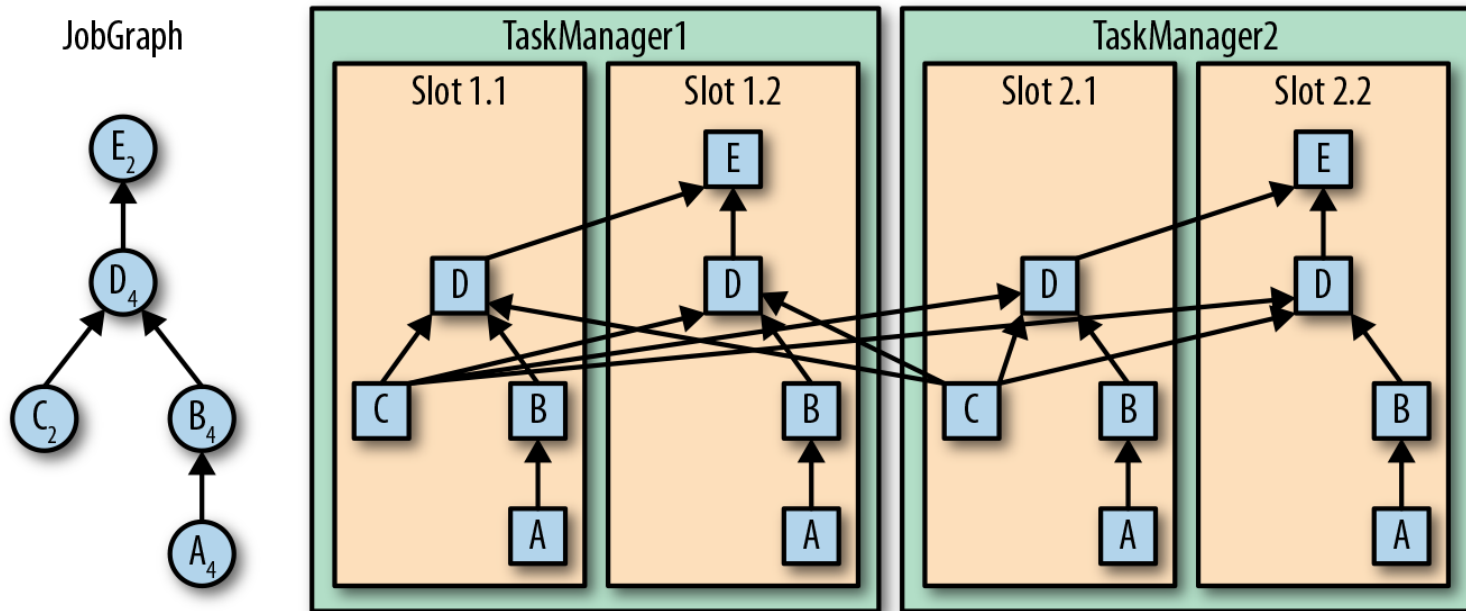
Task Execution

○ A TaskManager

- offers a certain number of processing slots to control the number of tasks it is able to concurrently execute.

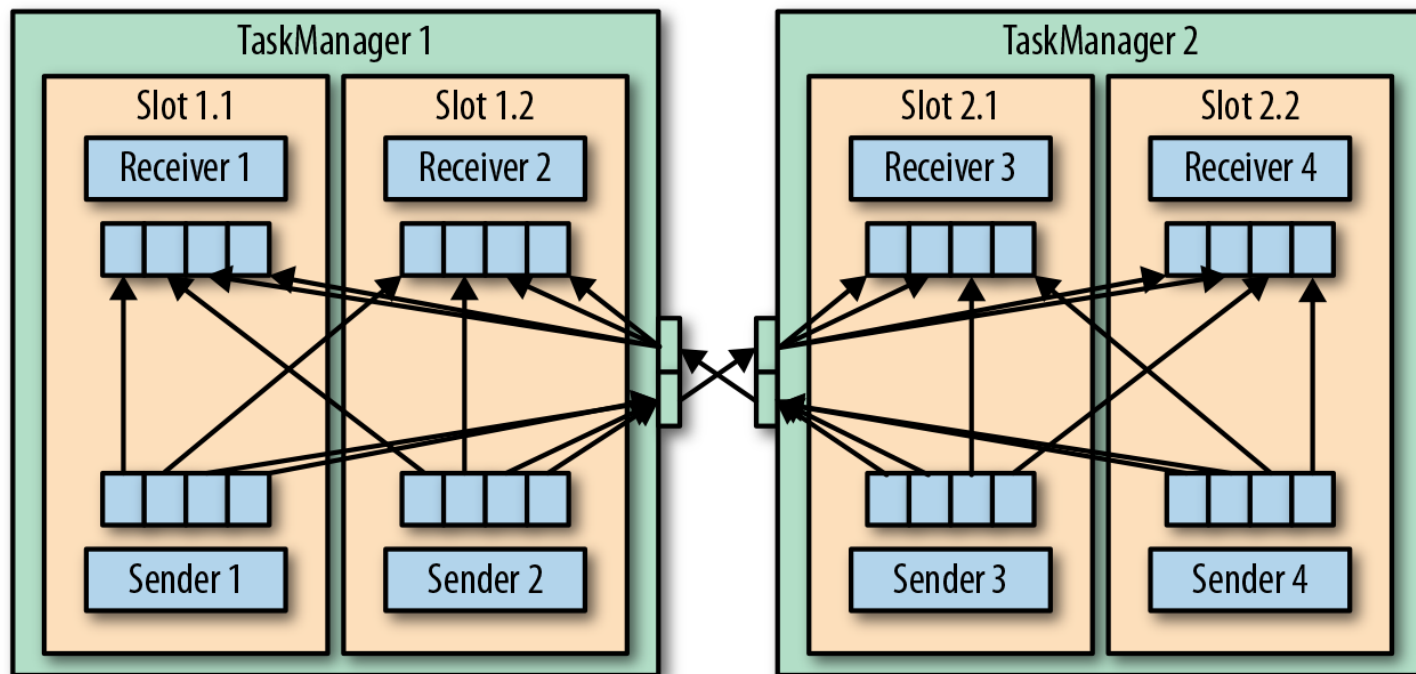
- A processing slot

- can execute one slice of an application—one parallel task of each operator of the application.



Data Transfer in Flink

- The tasks of a running application are continuously exchanging data.
- The TaskManagers take care of shipping data from sending tasks to receiving tasks.
- The network component of a TaskManager collects records in buffers before they are shipped.



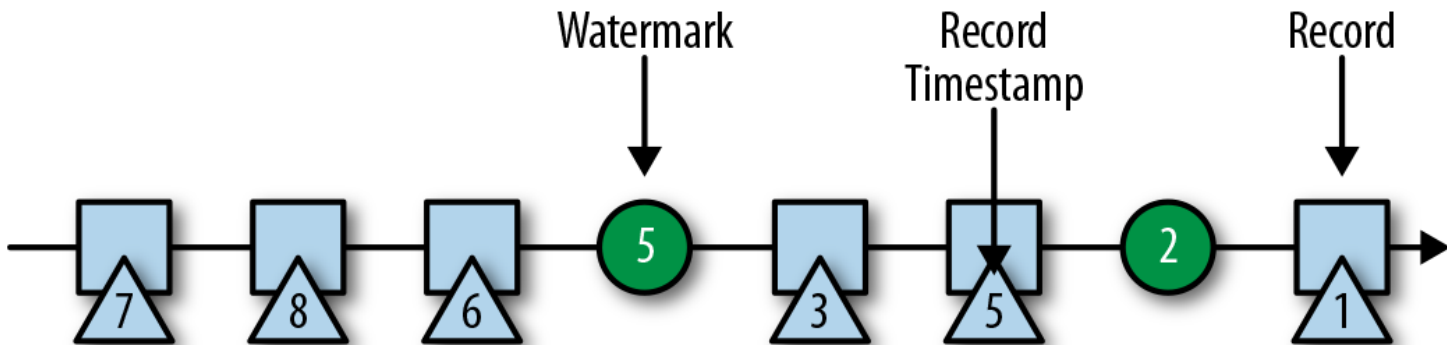
Event-Time Processing

○ Timestamps

- Every record must be accompanied by an event timestamp

○ Watermarks

- a Flink event-time application must also provide watermarks
- Watermarks are used to derive the current event time at each task in an event-time application
- In Flink, watermarks are implemented as special records holding a timestamp as a Long value. Watermarks flow in a stream of regular records with annotated timestamps.



Watermarks in Flink

- A Watermark is a special record that flows through the stream to signal **Event Time** progress.
 - **Concept:** A Watermark with timestamp T tells Flink: "I'm reasonably sure no more events with a timestamp $\leq T$ will arrive."
 - **Trigger:** When a Watermark of time T reaches a window operator, it triggers the calculation for any window ending at or before T .
 - **Problem:** In distributed systems, data arrives out of order due to network lag or partitions. Watermarks allow Flink to "wait" for a specified amount of time before closing a window.

Watermark Strategies

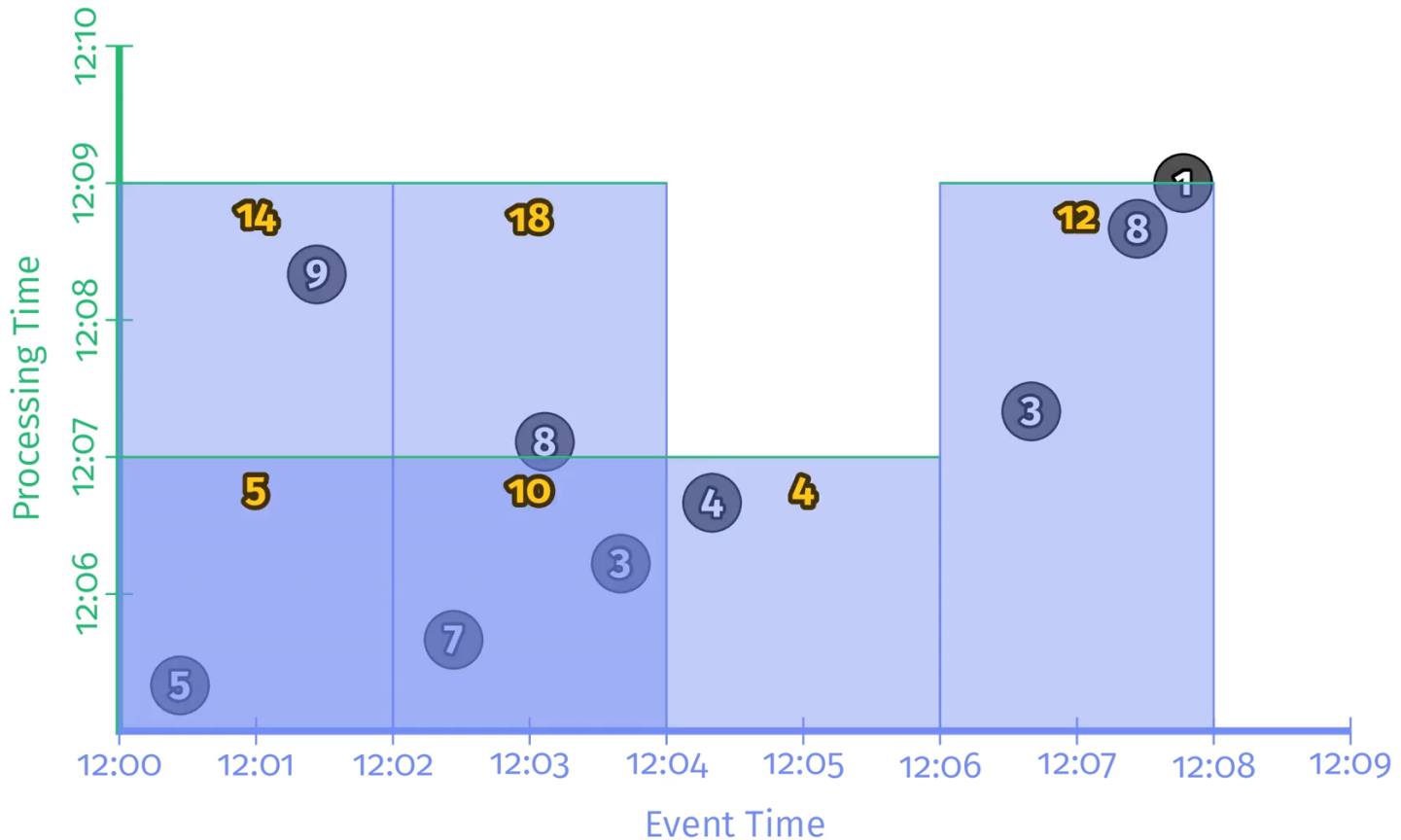
- Flink provides built-in strategies to handle different levels of data "messiness."

Strategy	Usage	Logic
Monotonous	Perfect ordering	Watermark = Current Max Timestamp.
Bounded Out-of-Orderness	Real-world streams	Watermark = Max Timestamp observed - L (fixed delay).
Custom	Complex logic	You define when and how the watermark advances

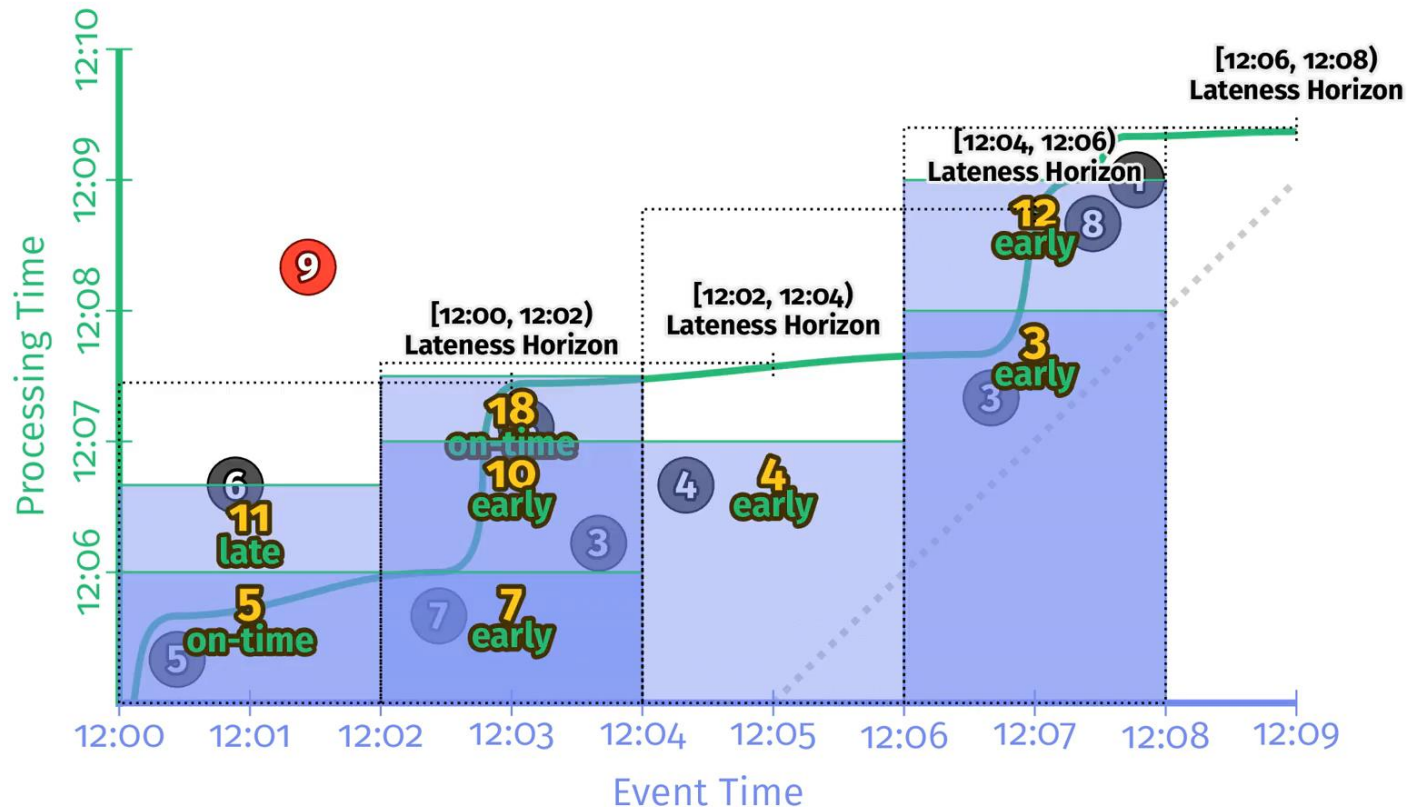
Handling Late Events

- Even with a L buffer (e.g. 5 seconds) , some data might arrive even later. Flink offers three ways to handle these "stragglers":
 - **Default (Discarding):** Once the watermark passes the window end-time, any late event is simply dropped.
 - **Allowed Lateness:** Keeps the window state around for an extra duration. If a late event arrives, the window is **re-fired** to update the result.
 - **Side Outputs:** The ultimate safety net. Late events are diverted to a separate stream for manual inspection or separate processing.

A Simple Counting Example (Spark with micro batches)



A Simple Counting Example (Flink with watermark records)

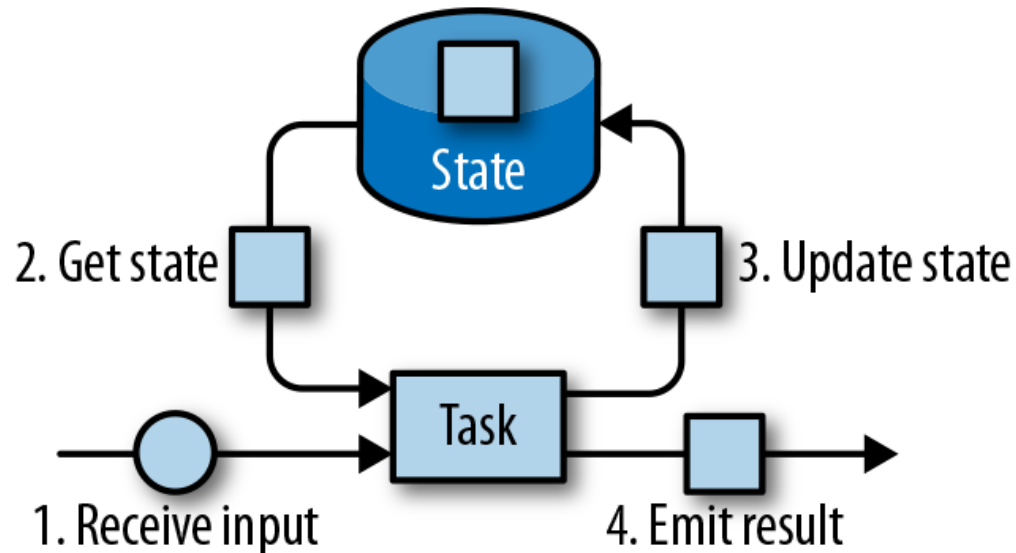


Heuristic watermark: —————

Ideal watermark:

State Management

- A stateful stream processing task
 - all data maintained by a task and used to compute the results of a function belong to the state of the task



State Management

○ State Backend

● Local State Management

- a task of a stateful operator typically reads and updates its state for each incoming record.
- each parallel task locally maintains its state in memory to ensure fast state accesses

● Checkpointing

- A TaskManager process may fail at any point, hence its storage must be considered volatile
- Checkpointing the state of a task to a remote and persistent storage
- The remote storage for checkpointing could be a distributed filesystem or a database system

Checkpoints

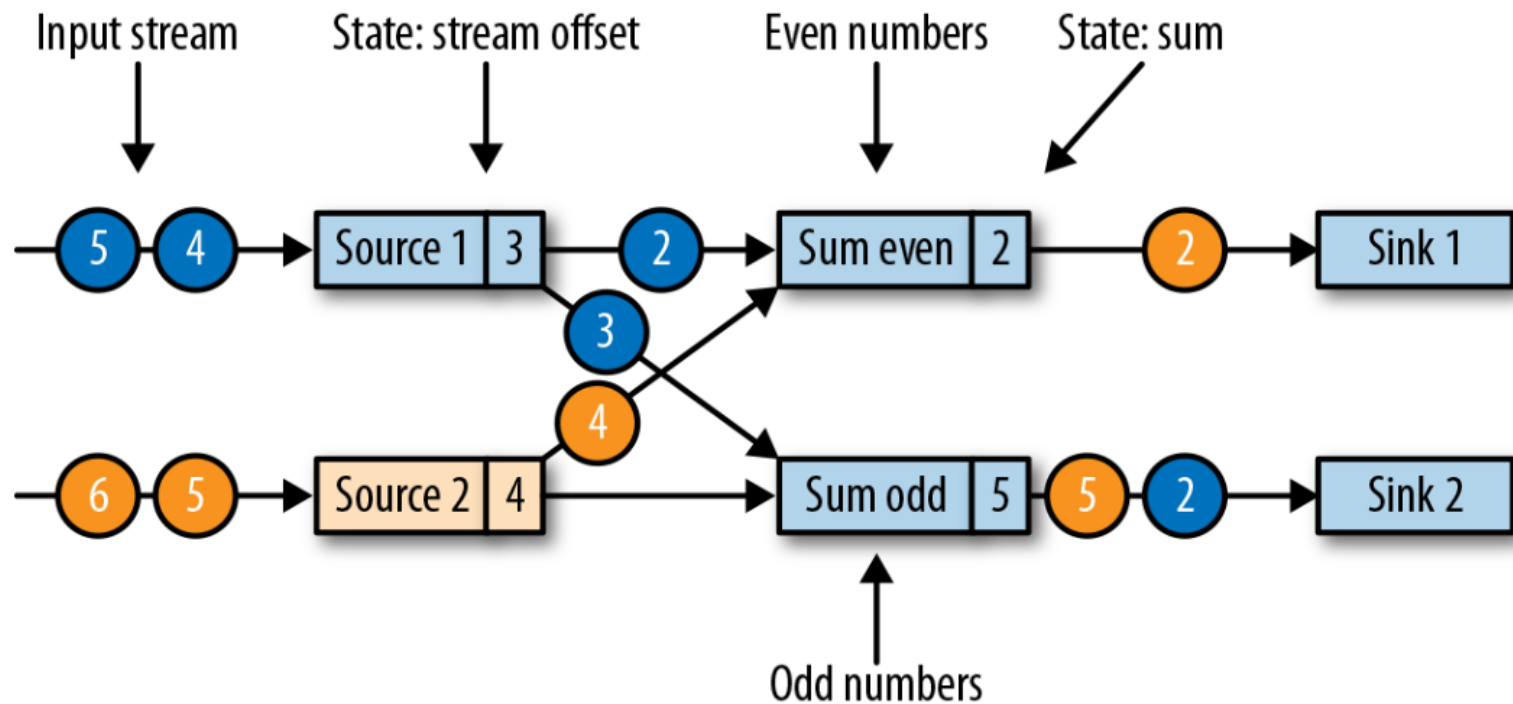
○ Flink's Checkpointing Algorithm

- based on the Chandy–Lamport algorithm for distributed snapshots
- does not pause the complete application but decouples checkpointing from processing
- some tasks continue processing while others persist their state

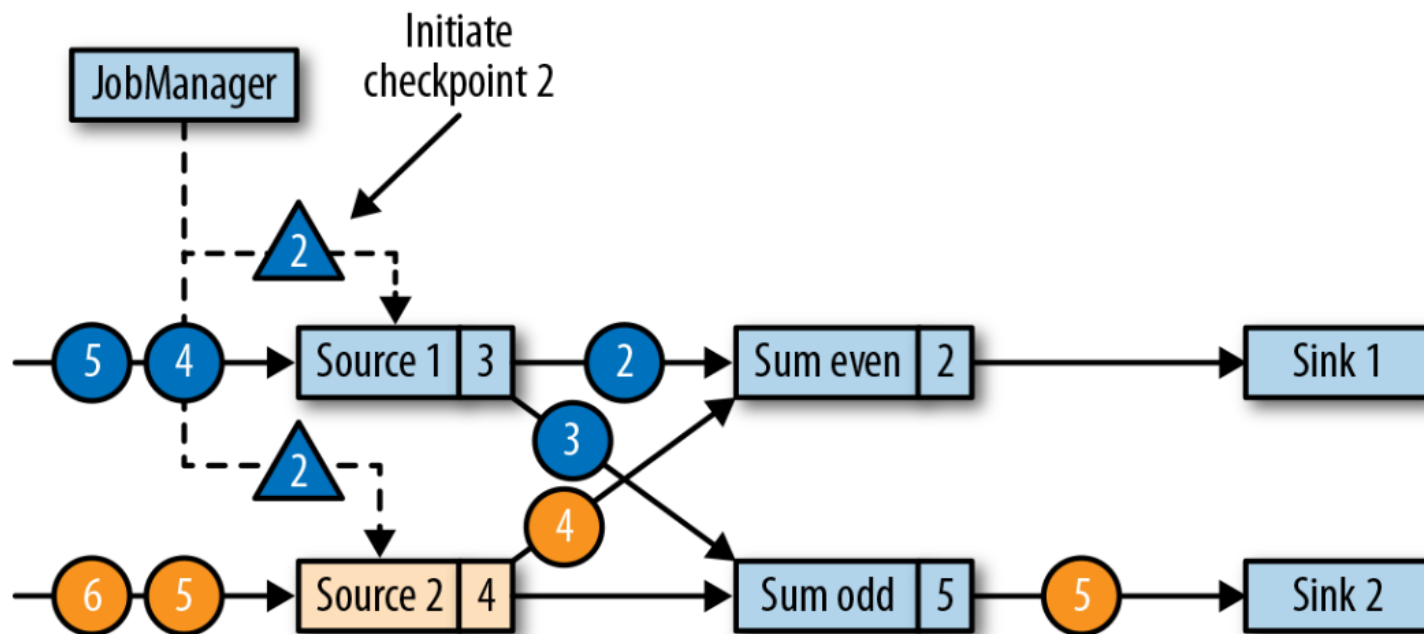
○ More details

- uses a special type of record called a *checkpoint barrier*
- checkpoint barriers are injected by source operators into the regular stream of records and cannot overtake or be passed by other records
- A checkpoint barrier carries a checkpoint ID to identify the checkpoint it belongs to and logically splits a stream into two parts
- All state modifications due to records that precede a barrier are included in the barrier's checkpoint and all modifications due to records that follow the barrier are included in a later checkpoint.

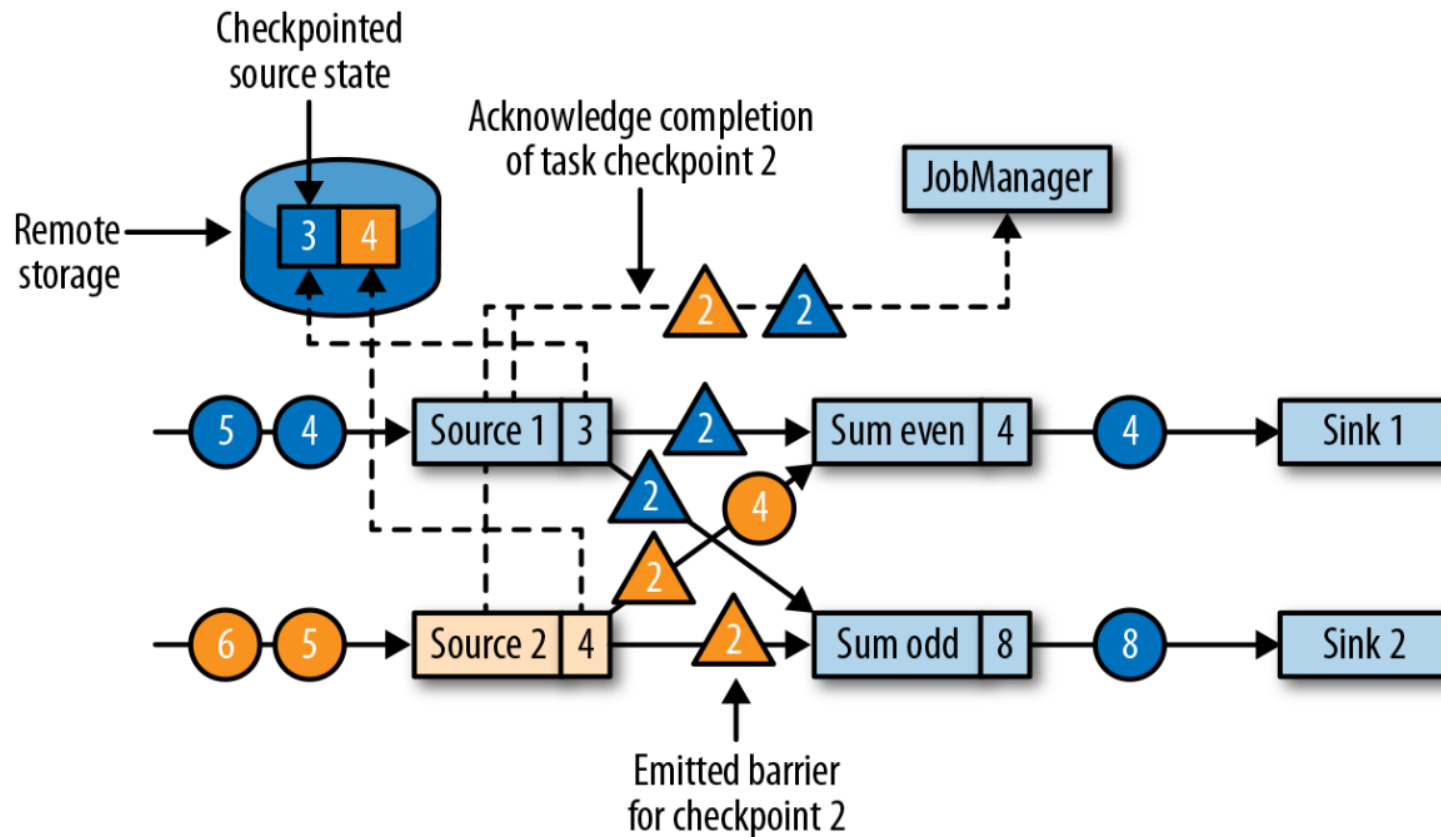
- Streaming application with two stateful sources, two stateful tasks, and two stateless sinks



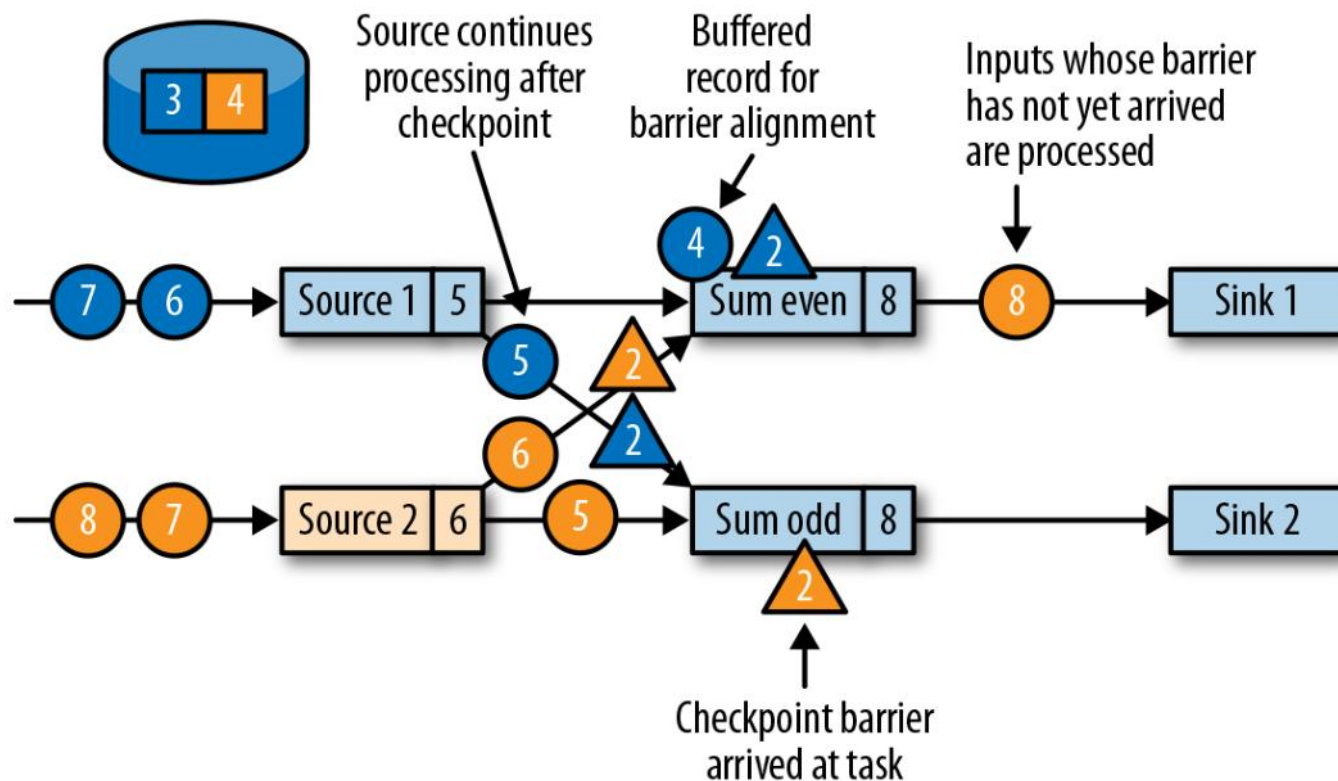
- JobManager initiates a checkpoint by sending a message to all sources



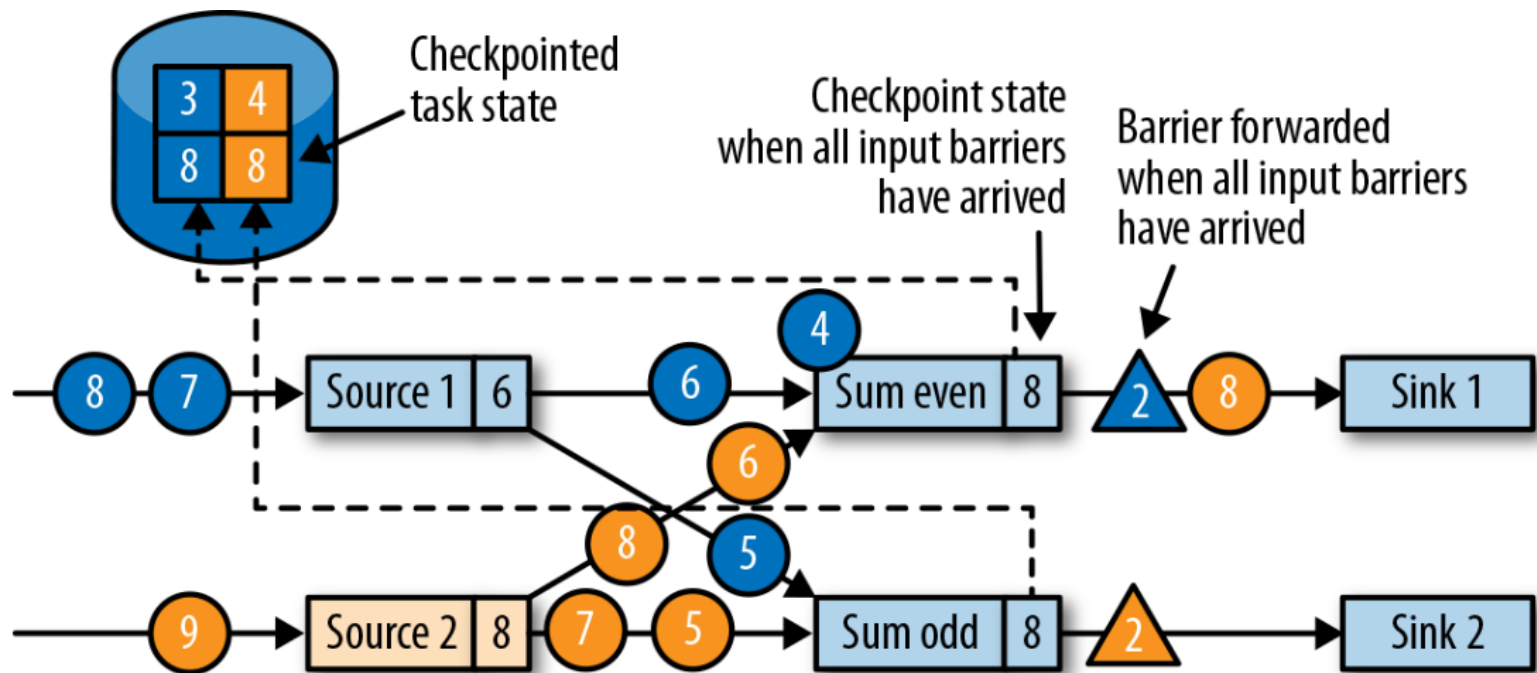
- Sources checkpoint their state and emit a checkpoint barrier



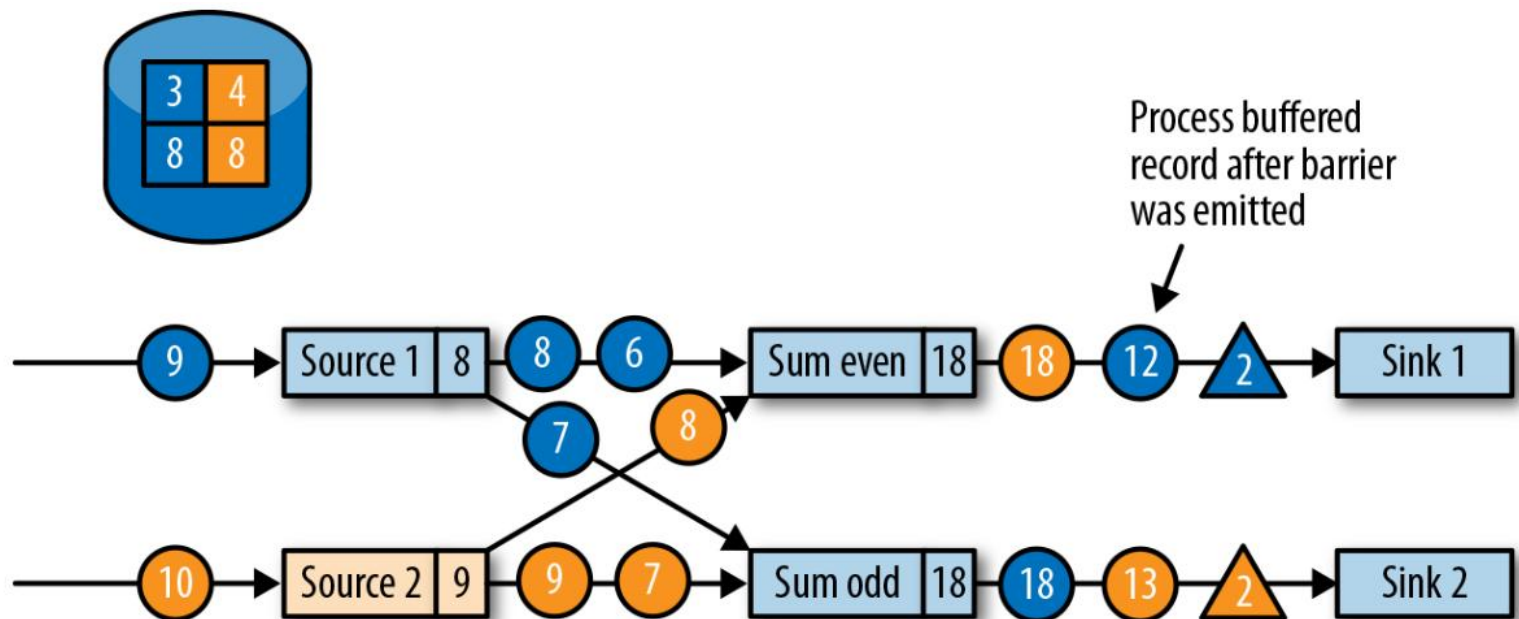
- Tasks wait to receive a barrier on each input partition
- Records from input streams for which a barrier already arrived are buffered
- All other records are regularly processed



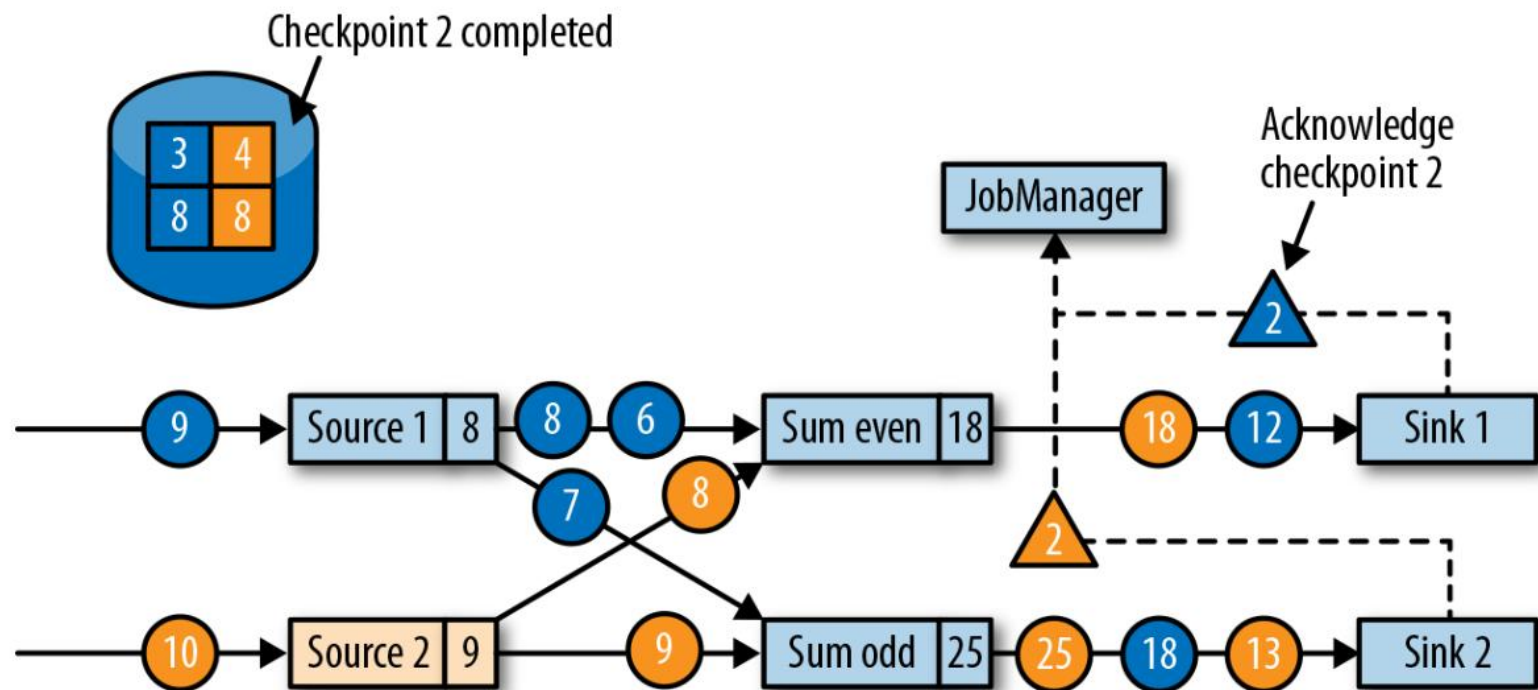
- Tasks checkpoint their state once all barriers have been received, then they forward the checkpoint barrier



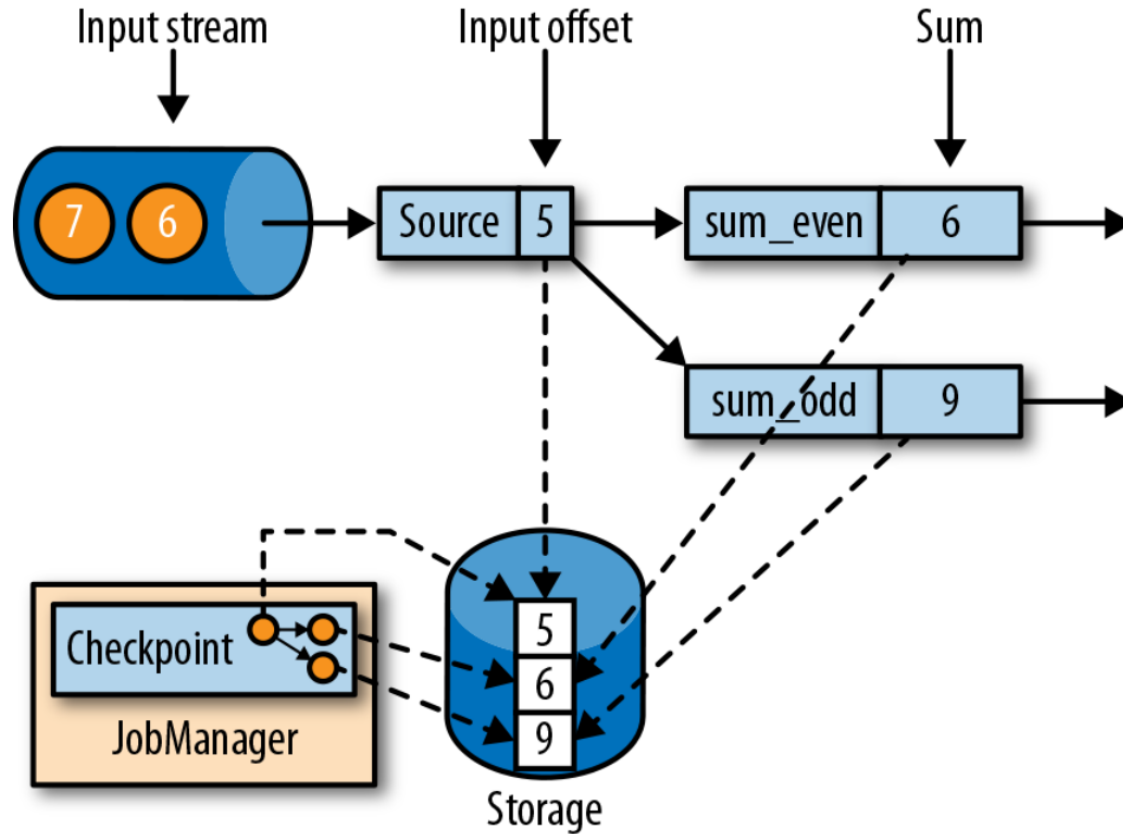
- Tasks continue regular processing after the checkpoint barrier is forwarded



- Sinks acknowledge the reception of a checkpoint barrier to the JobManager
- a checkpoint is complete when all tasks have acknowledged the successful checkpointing of their state



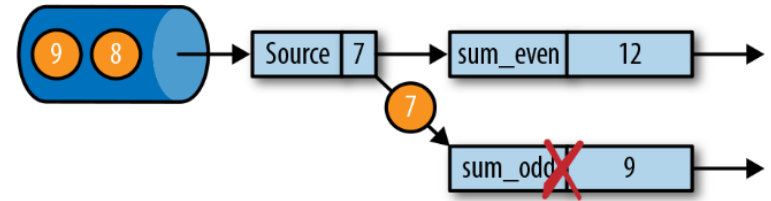
Failure Recovery from a checkpoint



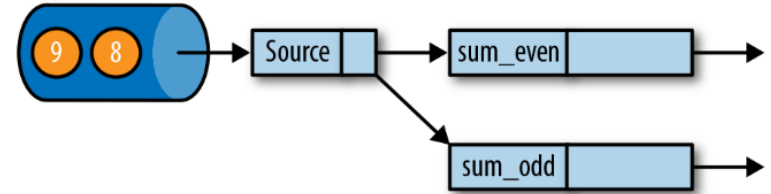
Failure Recovery

1. Restart the whole application.
2. Reset the states of all stateful tasks to the latest checkpoint.
3. Resume the processing of all tasks.

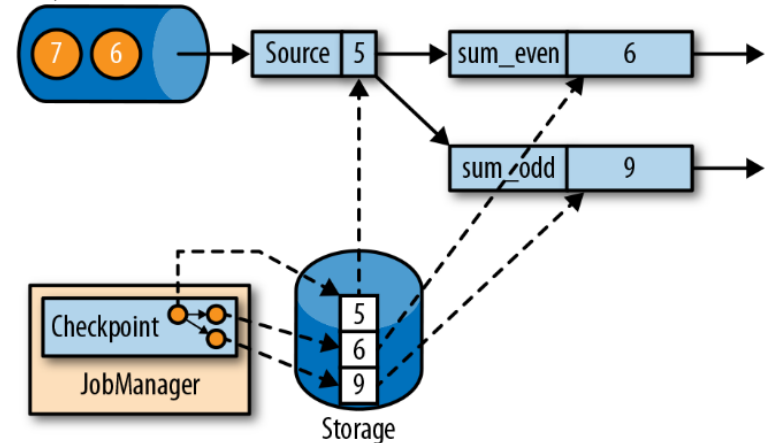
Failure: Task sum_odd fails



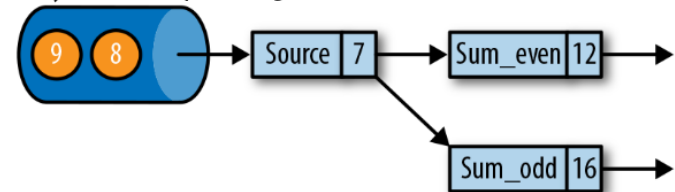
Recovery 1: Restart application



Recovery 2: Reset application state from Checkpoint



Recovery 3: Continue processing



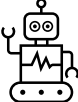
Savepoints

- A Savepoint is a user-triggered, consistent snapshot of a running Flink job's state.
 - **Concept:** Unlike Checkpoints (which are automatic and used for failure recovery), Savepoints are **manually triggered** by the operator.
 - **Contents:** It contains the state of all operators (e.g., window contents, hash maps) and the current read positions (offsets) in your sources (e.g., Kafka).
 - **Portability:** Savepoints are designed to be portable across different Flink versions and even different cluster configurations.

Savepoints Applications

- **Application Upgrades:** Take a savepoint, stop the job, update your JAR file, and resume from the savepoint.
- **Scaling:** If you need to change the **parallelism** (e.g., from 10 to 50 workers), you must restart from a savepoint so Flink can redistribute the state.
- **Cluster Migration:** Move a job from an on-premise cluster to the cloud by pointing the new job at the savepoint file path.
- **A/B Testing:** Start two different versions of the same job from the exact same point in time to compare performance.

Take-away in the AI Era

- (Why Flink Stream design still require humans)
-  **What AI cannot decide for you (yet)**
 - Watermark Strategy and Lateness Tolerance
 - How long do you wait for a late event? AI can't know if your business value disappears after 5 seconds of delay or if you must wait 2 hours for a stray sensor log to ensure financial accuracy.
 - Snapshot and Savepoints Strategy
 - how frequently to trigger a snapshot?
 - When to trigger a savepoint of the whole system?
-  **What AI can assist you**
 - Writing Complex Windowing Logic
 - State TTL (Time-To-Live) Configuration
 - AI can analyze your data patterns and suggest a State TTL configuration to automatically expire old states, preventing your state data from bloating indefinitely.

Conclusion: a comparison between Spark vs. Flink

○ Spark

- Microbatch streaming processing (latency of a few seconds)
- Checkpoints are done for each microbatch in a synchronous manner
- Watermark: a configuration to let user determine how to handle the late events

○ Flink

- Real-time streaming processing (latency of milliseconds)
- Checkpoints are done distributedly in an asynchronous manner (more efficient → lower latency)
- Watermark: a special record to let user estimate the current event time
 - Flink uses late handling functions together with watermarks to handle the late events

Acknowledgements

- CS4225 slides by He Bingsheng and Bryan Hooi
- Jules S. Damji, Brooke Wenig, Tathagata Das & Denny Lee, “Learning Spark: Lightning-Fast Data Analytics”
- Bill Chambers, Matei Zaharia, “Spark: The Definitive Guide”
- Fabian Hueske and Vasiliki Kalavri, “Stream Processing with Apache Flink”, 2019
- Tyler Akidau, Slava Chernyak and Reuven Lax, “Streaming Systems”, 2018